Graduate Certificate in Intelligent Reasoning Systems
Practice Module Final Report

The WellnessBot

By
The Stacks (Group 7)

Loo Kin Xing | Sherwin Tang Wei Hao | Foo Chit Yan | Foo Swee Yoong
A0073762B | A0125052W | A0341007W | A0341027R

Institute of System Science
National University of Singapore

Abstract

Discharged patients undergoing post-arthroscopic knee rehabilitation often lack continuous clinical support, increasing the risk of improper recovery and unsafe exercise practices. This project presents WellnessBot, a home-based virtual decision-support assistant designed to provide safe, personalized rehabilitation guidance and self-care recommendations.

The system adopts a neuro-symbolic approach, combining a structured knowledge graph and rule-based reasoning to ensure deterministic and explainable decision-making. Unlike purely LLM-based systems, WellnessBot follows a decision-first workflow, where recommendations are generated based on explicit clinical rules and constraints. A retrieval-augmented generation (RAG) module is incorporated to provide evidence-based explanations, while the large language model is restricted to natural language understanding and explanation. This ensures that all critical decisions remain controlled, transparent, and auditable.

The system is evaluated using a multi-layer framework, including decision-level classification performance and recommendation-level validation based on rehabilitation phase and user condition.

Overall, WellnessBot demonstrates a safe, interpretable, and modular approach to rehabilitation support, bridging the gap between research and real-world application while enabling continuous improvement through structured and traceable system refinement.

Acknowledgement

We would like to express our sincere gratitude to Dr. Gary Leung for his guidance and support throughout this project. His insights and suggestions have significantly contributed to the development of the system and the overall quality of this work.

We also extend our appreciation to the professors who taught the Machine Reasoning, Reasoning Systems, and Cognitive Systems modules, whose instruction and expertise have been invaluable in supporting this project.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Overview

This project develops a system to support recovery after post-arthroscopic knee surgery, providing safe and easy-to-understand exercise and self-care recommendations through a chat interface.

The system collects key information such as recovery stage, pain level, and symptoms, and uses it to determine the user's condition and provide appropriate exercise and self-care recommendations. Each recommendation is supported with clear explanation and evidence, helping user understands the reasonings behind it while ensuring transparency and reliability. The system also includes a feedback and improvement loop, where user interactions are analysed to identify issues and continuously improve future recommendations.

The scope focuses on post-arthroscopic knee surgery as a representative rehabilitation scenario, allowing the system design to be developed and evaluated effectively within project timeframe.

1.2 Importance & Relevance

Post-arthroscopic knee rehabilitation requires safe, stage-appropriate guidance, yet patients often lack clear and reliable support outside clinical settings. AI is a powerful tool for rehabilitation support, but for patients to feel confident using it at home, the reasoning behind each recommendation must be clear and easy to understand.

The novelty of this project lies in its transparent, decision-first approach, where recommendations are made using explicit rules and structured knowledge instead of opaque models, with the language model used only for interpretation and explanation. It also introduces an offline knowledge loop, where interaction logs and user feedback are analysed to refine rules and improve the system in a controlled and traceable manner.

1.3 Goal & Objectives

The goal of this project is to develop a system that provides safe, personalized, and easy-to-understand exercise and self-care recommendations to support recovery for post-arthroscopic knee surgery patients.

To achieve this, the system is designed with following objectives:

- To be able to understand user inputs through a chat-based interface, assess the user's recovery condition, and generate reliable and appropriate recommendations.
- To deliver clear and easily understandable explanations to user
- To be able to analyse user's feedback and continuously improve its recommendations over time.

2 Project Background & Market Research

2.1 Background

The growing demand for physiotherapy services in Singapore is not adequately met due to a shortage of physiotherapists, creating gaps in timely access to physio care (Insights10, n.d.).

In addition, the high cost of physiotherapy service, ranging from $50 to $400 per session, can pose a financial burden for many patients (Insights10, n.d.).

For individuals with mobility issues, accessibility is also a challenge for them as they may find it difficult to travel from home to treatment centres.

On top of that, patients often face difficulties in obtaining timely clinical advice, as support hotlines are frequently engaged, limiting their immediate access to professional guidance.

2.2 User Needs & Market Trends

There is a growing demand for digital healthcare solutions that address key user needs, including access to remote guidance without the need for in-person visits, real-time advice that is available anytime and anywhere, and personalized therapy that adapts to individual recovery progress (Abdul Wasay, 2026).

At the same time, emerging market trends are shaping the development of such systems.

Regulatory frameworks like the EU AI Act (2024) are driving the need for explainable and transparent clinical AI, while Asian Bioethics Review (2025) also highlights that explainability is one of the most important factors influencing user acceptance of AI in healthcare.

In addition, local regulations such as the Personal Data Protection Act in Singapore introduce stricter guidelines for data protection and AI system governance, further emphasizing the importance of building trustworthy, compliant, and user-centric wellness solutions.

2.3 Market Overview and Opportunities

According to Artificial Intelligence (AI) Enhanced Remote Physiotherapy Market Report 2026 (Abdul Wasay, 2026), AI Enhanced Remote Physiotherapy market size has reached to $2.12 billion in 2025 and it is expected to grow to $6.44 billion in 2030 at a compound annual growth rate (CAGR) of 24.8%, with North America was the largest region in 2025 and Asia-Pacific is the fastest growing region in forecast (Abdul Wasay, 2026).

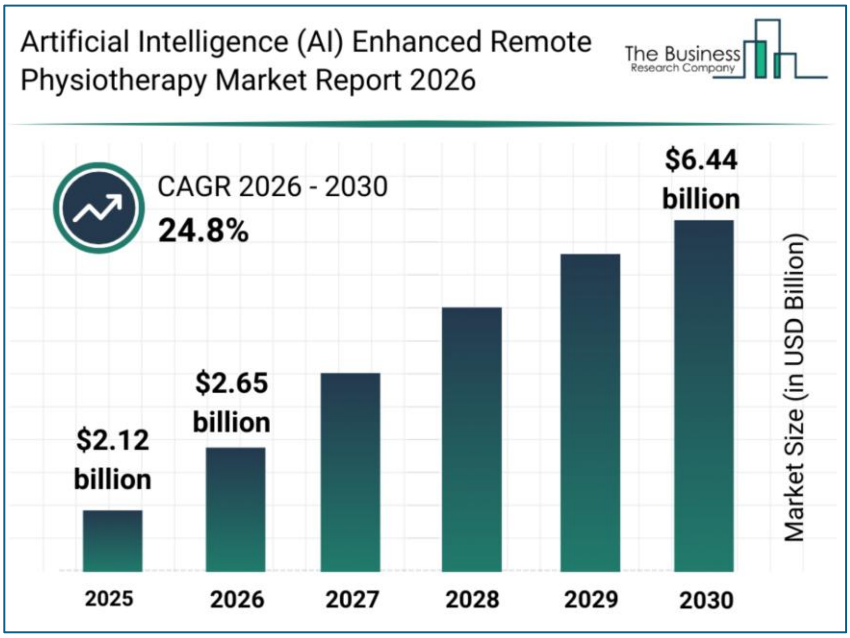


Figure 1: Global Artificial Intelligence (AI) Enhanced Remote Physiotherapy Market Size Gain ($ Billion), 2025 – 2030

The AI enhance remote physiotherapy market can be divided by five segments (Abdul Wasay, 2026) and table below shows how WellnessBot is positioned in each segment.

Market Segment	WellnessBot
By Component	Software
By Therapy	Musculoskeletal
By Deployment	In-App (non-cloud)
By Application	Rehabilitation
By End-User	Homecare settings

Table 1: WellnessBot in Market Segment

From the market segments listed above, it is clearly seen that WellnessBot has the potential opportunities to incorporates other types of musculoskeletal injuries such as ligament tears or rheumatoid arthritis. It can also be further developed for other application such as ergonomic exercise and other user settings such as fitness centre.

2.4 Key Players Comparison

There are plenty of key players in AI enhanced remote physiotherapy market, and table below shows some of the comparisons of WellnessBot versus these key players.

Key Player	**How it is compared to WellnessBot**
Hinge Health which focuses on rehabilitation AI	No post-surgical constraint filtering
Physitrack which provides exercise libraries	No conversational AI or safety filtering
Sword Health which provides AI physical therapy	Use of wearable sensors unlike device-free WB
Kaia Health which focuses on digital treatment for musculoskeletal conditions	Use of camera which might lower user's acceptance

Table 2: Comparing WellnessBot with other Key players

3 Project Scope

3.1 Academic

This system demonstrates all four AI systems paradigms from course criteria in a single integrated system:

- Decision Automation → Rule & Inference Engine (RECOMMEND / FORBID / ESCALATE)
- Business Resource Optimisation → Intervention Planner (weighted multi-criteria scoring as informed search)
- Knowledge Discovery → LLM semantic extraction, RAG retrieval, structured interaction logs
- Cognitive / Knowledge Systems → Knowledge Graph, User State Inference, LLM response generation, full chatbot

It contributes a two-layer safety design pattern (hard constraint pre-filter + soft scoring), which is also applicable to other safety-critical recommendation domains.

The audit log bridges rule-based explainability which is academically relevant to clinical AI governance debate.

3.2 Clinical

From clinical domain perspective, this application targets:

- single surgery which is arthroscopic knee surgery
- adult user who is 0 to 20 weeks post-operation which is a standard recovery trajectory
- six weeks of rehabilitation phases - Initial, Early, Intermediate, Late, Transitional and Return-to-Sport
- single best exercise recommendation per session

3.3 Limitations

The system has several limitations that should be acknowledged. The knowledge graph is pre-populated and does not support real-time updates, requiring manual intervention whenever clinical guidelines change.

In addition, the system does not incorporate online learning; instead, user interaction logs are stored for future offline analysis and improvement.

Furthermore, the system relies on subjective and unverified user inputs, which could affect the reliability of recommendations.

Recovery progression is also determined using a time-based phase mapping approach rather than criterion-based assessment.

4 Data Collection and Preparation

4.1 Data Collection

The knowledge base for WellnessBot is constructed from published clinical rehabilitation guidelines obtained from reputable sources, including hospitals, medical centres, and clinical professionals.

The system focuses specifically on post-arthroscopic knee surgery rehabilitation, where the overall protocol structure is primarily derived from the Massachusetts General Hospital rehabilitation guideline. Additional clinical sources are referenced to supplement detailed exercise instructions and self-care recommendations.

Unlike data-driven machine learning systems, the knowledge in WellnessBot is explicitly curated and structured to reflect clinically accepted practices. Each knowledge element, including rehabilitation phases, exercises, and safety rules, is associated with source references to ensure traceability and support evidence-based reasoning.

4.2 Data Preparation

The knowledge is initially organised in spreadsheet format to facilitate systematic population and validation based on domain understanding. During this stage, rehabilitation phases, exercises, constraints, and supporting evidence are manually structured and standardised.

Subsequently, the knowledge is transformed into a structured YAML-based representation, forming the system's knowledge graph. Each entity, such as phases and exercises, is assigned unique identifiers to enable deterministic rule evaluation and planner integration.

In addition, evidence content is segmented into smaller instruction chunks and linked to their respective exercises, allowing controlled retrieval during the explanation stage. Supporting data, such as interaction logs and audit traces, are maintained in JSON format to enable traceability and offline knowledge loop analysis.

Figure 2 illustrates the transformation of unstructured clinical rehabilitation guidelines into a structured YAML-based knowledge graph. Clinical knowledge is first organised into structured tables to define rehabilitation phases, exercises, safety rules, and supporting evidence. These elements are then mapped into a unified knowledge graph representation, enabling deterministic rule evaluation, constraint-aware exercise planning, and traceable evidence-based explanations.

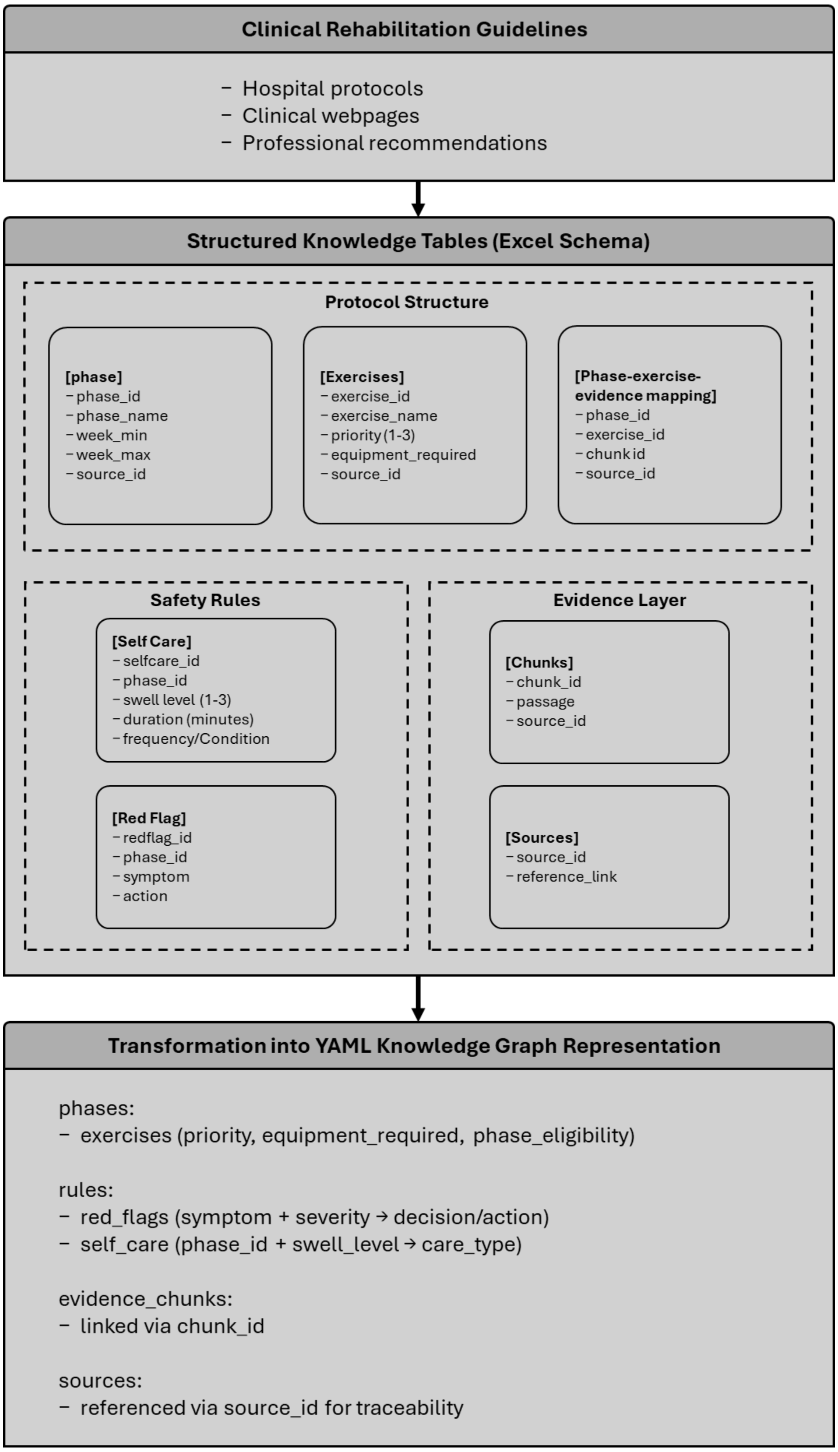


Figure 2: Knowledge Structuring and Transformation into YAML-Based Knowledge Graph

4.3 Dataset Overview

Below is the overview of current dataset structure. This structured knowledge base defines the scope and operational constraints of the system, enabling deterministic reasoning across rehabilitation scenarios.

- Number of Protocol: 1
- Phases per Protocol: 6 (where final phase is defined as Return-to-Sport and no step-by-step instruction will be given)
- Exercise candidates: 25
- Instruction chunks per exercise: 1 to 2
- Exercise priority rating: 1 to 3 (where 1 must be prioritized first)
- Equipment options: Stool, Chair, Towel, Resistance band, Bicycle etc
- No. of decision rules: 20 (including red flag and self-care)
- Test scenarios: 16 Recommend, 7 Forbid, 9 Escalate, with 11 user inputs generated using 2 LLM models (Gemini 3 Flash & Claude Sonnet 4.6)

4.4 Dataset Challenges

Several challenges were encountered in the development of the system, along with corresponding mitigation strategies.

First, the definition of rehabilitation phases varies across clinical guidelines, which may introduce inconsistencies in protocol design. To address this, a single standardised protocol from a reputable hospital was adopted as the primary reference to ensure consistency.

Second, patient physical conditions may differ even within the same recovery phase. To better capture this variation, the early stage of rehabilitation is further divided into sub-phases, with exercises explicitly labelled and prioritised to reflect differing levels of difficulty and suitability.

Third, general self-care recommendations are often dispersed across multiple sources and presented in unstructured formats. These were consolidated and transformed into structured rules based on recovery phase and symptom severity, enabling consistent and interpretable guidance.

Finally, many rehabilitation exercises include progressive variations that may not be universally safe for all users. As a precaution, progressive recommendations are excluded to prioritise safety and reduce the risk of inappropriate exercise selection.

A key limitation of the system is the absence of real patient datasets. However, this is an intentional design choice, as the system prioritises deterministic, explainable, and safety-critical decision-making over data-driven learning. In clinical decision-support settings, reliance on curated guidelines and explicitly defined rules provides greater transparency, consistency, and auditability compared to purely data-driven approaches.

4.5 Scenario Design and Test case construction

The evaluation scenarios were designed to reflect realistic rehabilitation conditions while ensuring systematic coverage of key system components. The design was guided by the following considerations:

- **Robustness of NLU extraction:** Two LLM backends, Claude Sonnet 4.6 and Gemini 3 Flash (Fast), were used exclusively for schema-constrained extraction to reduce reliance on any single model and mitigate model-specific bias in language interpretation. As the LLM is strictly limited to NLU, consistent downstream decisions across both models confirm that system reasoning remains independent of model behaviour.
- **Domain-driven design:** All scenarios and expected outcomes were defined based on domain knowledge. The distribution of cases was intentionally constructed to ensure coverage of all decision outcomes, including RECOMMEND, FORBID, and ESCALATE, enabling systematic validation of both routine and safety-critical behaviours.
- **Phase-dependent reasoning:** User profiles were varied through surgery dates to simulate different rehabilitation phases, ensuring that rules, exercise eligibility, and self-care protocols are applied appropriately.
- **Temporal consistency:** A single user profile was evaluated across multiple time points, allowing phase transitions to be tested and verifying that decisions remain coherent as recovery progresses.
- **Symptom variation:** Inputs were diversified across pain levels, swelling severity, and red-flag conditions (e.g., fever, excessive bleeding) to evaluate system responses under different clinical states.
- **Constraint-based planning:** Equipment selection was controlled to validate that exercise recommendations remain feasible and aligned with user constraints.

These considerations ensure comprehensive coverage of the system's deterministic decision logic, safety mechanisms, and constraint-aware planning under realistic operating conditions.

5 System Design

5.1 Design Objectives and Requirements

The design of WellnessBot is guided by the need for safe, interpretable, and auditable rehabilitation support. The system must ensure that all recommendations are clinically conservative and derived from explicit logic rather than probabilistic generation. Key requirements include:

1. Deterministic decision-making to guarantee consistent outputs
2. Explainability through traceable rules and evidence
3. Auditability via structured logging of decisions and reasoning steps, and
4. Strict separation between reasoning and language processing to prevent uncontrolled LLM influence.

Additionally, the system must support personalization through user state and profile data, while maintaining no runtime learning, ensuring that all updates occur through controlled, offline refinement. These requirements collectively shape a modular, safety-first architecture suitable for real-world rehabilitation contexts.

5.2 Overall System Architecture

The architecture of WellnessBot is presented through three complementary views, capturing the system's decision flow, architectural layering, and lifecycle refinement.

Figure 3 illustrates the core decision pipeline. User input is first processed by an LLM for structured extraction, followed by user state inference. The inferred state is evaluated against a YAML-based knowledge graph through a deterministic rule engine, which produces one of three controlled actions: ESCALATE, FORBID, or RECOMMEND. When recommendation is permitted, planner selects an appropriate exercise. Explanation is generated only after the decision is finalized, using a retrieval-augmented approach.

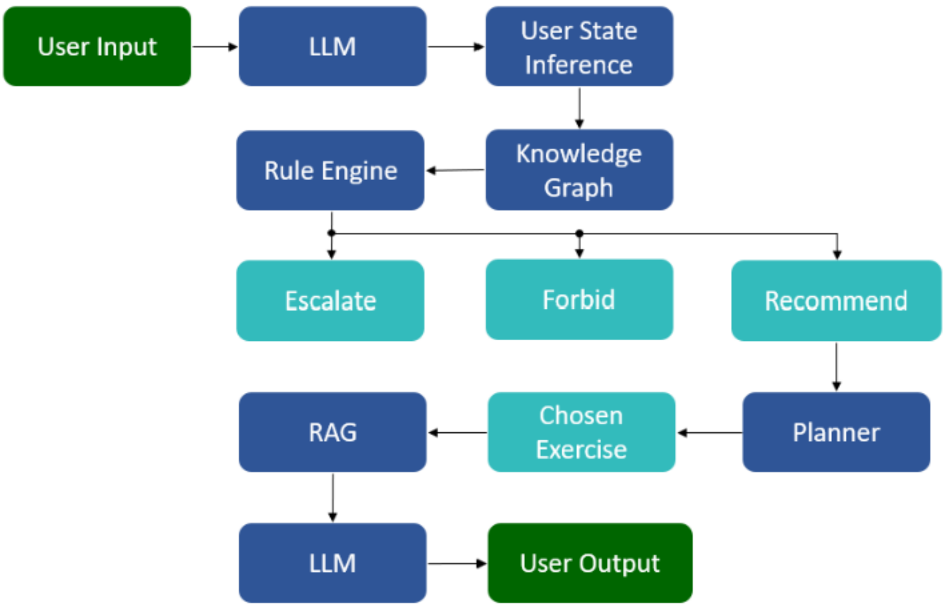


Figure 3: Core Decision Pipeline of WellnessBot

Figure 4 shows the offline knowledge loop, where interaction logs are analysed to identify rule gaps, knowledge inconsistencies, and planner limitations. Updates to the knowledge graph, rules, and planner are performed manually, ensuring that no runtime learning affects system behaviour.

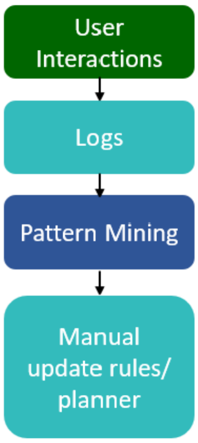


Figure 4: Offline Knowledge Loop for System Refinement

Figure 5 presents the layered architecture, separating the system into presentation, application, and data tiers. This modular design supports maintainability, auditability, and clear separation between user interaction, reasoning logic, and data storage.

Together, these views demonstrate a system that is not only functionally complete, but also structured for safe reasoning, transparency, and continuous improvement.

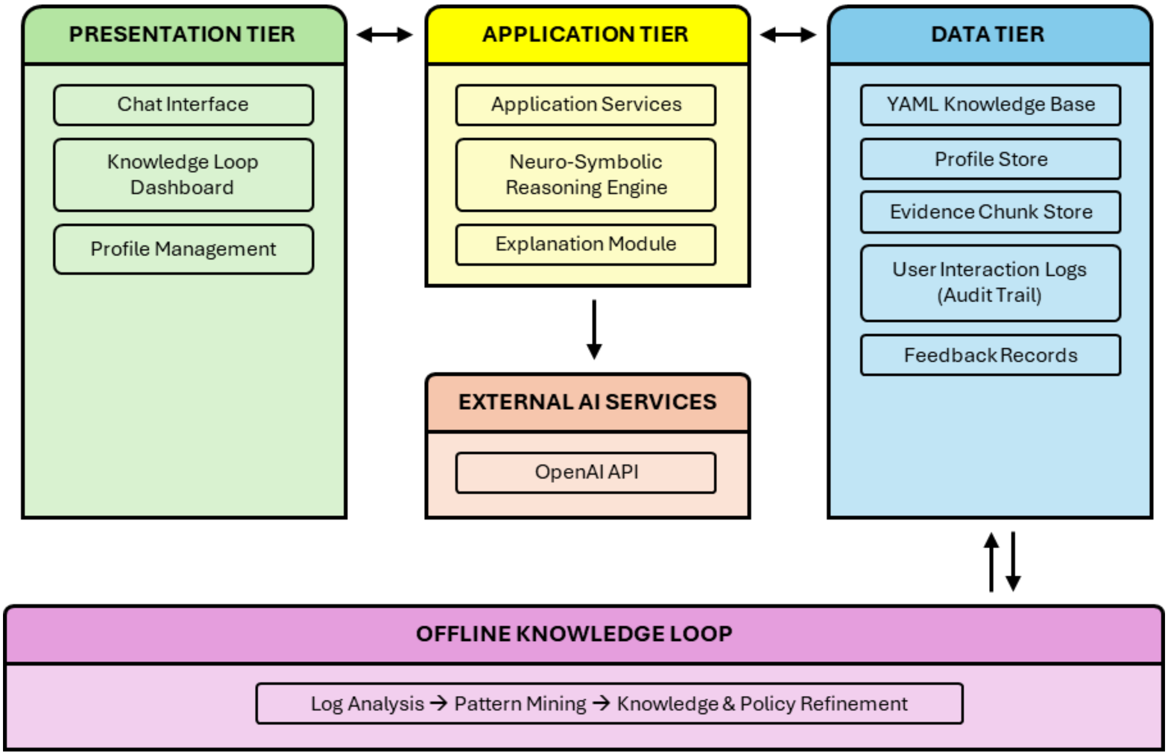


Figure 5: Layered System Architecture of WellnessBot

5.3 Reasoning Technique Selection and Design Rationale

WellnessBot adopts a neuro-symbolic approach to ensure safe, interpretable, and controllable decision-making. A Knowledge Graph (YAML) encodes rehabilitation protocols explicitly, enabling traceable and maintainable domain knowledge. A deterministic Rule Engine performs safety-critical reasoning, enforcing constraints such as phase eligibility and red-flag conditions with a fixed priority (ESCALATE > FORBID > RECOMMEND). A planner is applied only after safety is confirmed, selecting an appropriate exercise based on constraints such as equipment availability, exercise priority and recent history.

Fully LLM-based reasoning was not adopted due to its non-deterministic and non-auditable nature, which is unsuitable for safety-sensitive rehabilitation guidance. Instead, LLMs are restricted to structured extraction (NLU) and post-decision explanation (RAG). This ensures that all decisions are derived from explicit logic while retaining natural language interaction capabilities.

The effectiveness of this design is evaluated in a later section using a structured framework that assesses both decision correctness and recommendation validity.

5.4 Natural Language Understanding (NLU) Design

The NLU module in WellnessBot is designed as a controlled extraction layer that converts unstructured user input into a structured representation for downstream reasoning. An LLM is used to handle diverse natural language expressions, mapping them into predefined fields such *as pain_score, swelling_score, symptom_flags, red_flag_terms, surgery_type, surgery_date,* and *weeks_since_event* under a strict prompt and schema constraint.

A key function of the NLU module is the early identification of red flag symptoms from natural language descriptions. For example, inputs such as "bleeding" or "there is blood" are mapped to structured signals (e.g., *symptom_flags* = ["excessive_bleeding"] and *red_flag_terms* = ["excessive bleeding"]). The module also extracts temporal information, where explicit dates (e.g., "2026-04-01") are captured as *surgery_date* and converted into *weeks_since_event* for downstream phase inference.

The LLM performs input interpretation only, while all reasoning and decision-making are handled by deterministic system components. To ensure reliability, normalization and validation are enforced through controlled input mechanisms and deterministic post-processing. For instance, structured inputs such as pain and swelling levels are captured via predefined button selections (discussed in a later section), reducing ambiguity at source. Additional normalization handles cases such as "no fever", ensuring negated symptoms do not trigger incorrect red flag detection.

This design ensures that all inputs entering the reasoning pipeline are standardized, reliable, and safe for downstream decision-making.

5.5 Conversation State and User Profile Management

WellnessBot maintains a structured conversation state to support multi-turn interaction and ensure that all required information is explicitly collected before reasoning. Key fields such as *surgery_type, surgery_date, pain_score, swelling_score,* and *symptom_flags* are obtained through a guided and ordered questioning process, rather than inferred from incomplete input. Pain and swelling levels are always explicitly requested through controlled inputs unless the case is escalated due to red flag conditions.

In addition, a persistent user profile is maintained, storing information such as surgery type, surgery date, and available equipment. This profile is merged with the current conversation state to enable personalized and context-aware reasoning. Exercise history is also retained to avoid repetition in recommendations.

This design ensures that the system operates on a complete, consistent, and user-specific state, reducing ambiguity and preventing unsafe decisions caused by incomplete or unreliable inputs.

5.6 Knowledge Representation: YAML-Based Knowledge Graph

WellnessBot uses a YAML-based Knowledge Graph to represent rehabilitation protocols in a structured and interpretable form. Each protocol is defined by *surgery_type* and organized into phases with explicit week ranges (*week_min, week_max*), enabling deterministic phase inference based on recovery time. Within each phase, exercises are defined with attributes such as allowed_in, priority, equipment_required, and linked evidence references.

The knowledge graph also encodes exercise aliases for mapping user input to canonical exercise identifiers, red flag policies that define condition-specific actions based on symptoms and severity, and self-care actions parameterized by phase and symptom level. This allows both exercise recommendation and supportive care to be represented within the same structured protocol layer.

A YAML-based representation is chosen over graph databases because the system primarily requires structured protocol lookup and rule-based access, rather than complex graph traversal. This keeps the knowledge layer transparent, easy to maintain, and directly aligned with deterministic reasoning. As a result, decisions can be traced back to explicit protocol definitions while remaining simple to update and extend.

5.7 Deterministic Rule Engine Design

The Rule Engine is the core decision-making component of WellnessBot. It evaluates the structured user state against a fixed set of explicit rules and produces a controlled action. To preserve safety, rule evaluation follows a two-pass design. In the first pass, the engine collects all non-recommendation outcomes, allowing higher-priority actions such as escalation or prohibition to be identified before any recommendation is considered. If no such rule fires, a second pass evaluates recommendation rules.

When multiple rules are triggered, the final action is resolved using a predefined dominance order, ensuring that safer actions override more permissive ones. An ESCALATE result acts as a hard stop, while a FORBID outcome prevents unsafe recommendations while still allowing supportive guidance to be included.

This design ensures that decision-making remains deterministic, conservative, and auditable, with every outcome traceable to explicit fired rules.

5.8 Planner Design

The planner is responsible for selecting a suitable exercise after safety constraints are satisfied by the Rule Engine. It operates over candidate exercises retrieved from the knowledge graph for the current phase and applies a deterministic selection strategy based on priority, equipment availability, and user history.

The selection process follows three main stages. First, exercises are filtered based on equipment availability to ensure feasibility. Second, a priority-based progression mechanism is applied, where exercises are organized into tiers and selected sequentially within each tier while avoiding repetition using exercise history. Third, the planner incorporates adaptive behavior through pain-based adjustment, where mild pain triggers a downgrade in difficulty, including cross-phase fallback to earlier rehabilitation stages if necessary.

If all exercises within a phase are exhausted, the planner signals completion, allowing the system to escalate or transition appropriately. This design ensures that recommendations are progressive, personalized, and safe, while remaining fully deterministic and traceable.

5.9 Restricted Role of LLM and RAG Explanation Design

The LLM in WellnessBot is deliberately restricted to language-facing tasks only and is excluded from decision-making. Its role is limited to two functions: structured input extraction in the NLU module and final response verbalization after a decision has already been made. Explanation generation is grounded through a BM25-based retrieval pipeline, where evidence chunks are retrieved from a curated chunk store using planner-linked evidence chunks and chunk identifiers.

The final response is then generated from a structured payload containing the selected exercise, supportive care, and retrieved evidence. The response prompt explicitly forbids the model from changing the recommendation, adding new advice, or exposing internal logic such as rules or planner details. Validation is applied after generation to ensure the final explanation remains consistent with the selected exercise and permitted output structure.

This design preserves a strict separation between reasoning and language generation, ensuring that the system remains deterministic while still providing patient-friendly, evidence-grounded explanations.

5.10 Logging, Audit Trail, and Offline Knowledge Loop

WellnessBot implements a structured logging and audit trail mechanism to ensure full traceability of system behaviour. Each interaction log captures the original user input, extracted NLU output, final decision, associated rule identifiers, and supporting sources. In addition, a detailed audit trace records intermediate reasoning states, including inferred phase, detected risk flags, rule execution results, and contextual information such as block reasons and system state.

This design enables every decision to be fully explainable and reproducible, as the complete reasoning path can be reconstructed from logs. User feedback is recorded separately, capturing evaluation signals such as thumbs-up or thumbs-down responses.

These logs form the basis of an offline knowledge loop, where interaction data is analysed to identify rule weaknesses, knowledge gaps, and recurring failure patterns. Lightweight pattern mining is applied to both interaction and feedback logs to uncover recurring failure cases, rule conflicts, and user dissatisfaction patterns, supporting targeted refinement of the rule set and knowledge graph. All updates are performed manually, ensuring that system improvements remain controlled and do not affect runtime determinism.

5.11 Design Strengths and Trade-offs

The design of WellnessBot demonstrates several key strengths. First, the decision-first architecture ensures that all outputs are derived from deterministic logic, improving safety and consistency. The use of a YAML-based knowledge graph enables transparent and maintainable domain representation, while the Rule Engine provides explicit and auditable reasoning. The planner further enhances the system by introducing structured progression and personalization, ensuring recommendations remain appropriate to the user’s recovery stage. In addition, the strict separation between reasoning and LLM usage mitigates risks associated with hallucination and uncontrolled generation.

However, this design also involves trade-offs. The reliance on predefined rules and structured knowledge limits flexibility and requires manual updates to handle new scenarios or expand coverage. The deterministic nature of the system may reduce adaptability to highly complex or ambiguous cases. Furthermore, the use of controlled input mechanisms and structured flows may reduce conversational flexibility compared to fully generative systems.

Overall, the design prioritizes safety, transparency, and controllability over flexibility, making it well-suited for rehabilitation support where correctness and auditability are critical.

6 Implementation

6.1 System Implementation Overview

WellnessBot is implemented as a modular Python-based system, with clear separation between language processing, symbolic reasoning, and user interaction. The application is built using Streamlit for the user interface, enabling interactive profile management, guided input collection, and transparent display of system outputs. Rehabilitation knowledge is stored in YAML-based protocol files, while user profiles, interaction logs, and feedback records have persisted in JSON format for traceability and audit.

The core reasoning pipeline is implemented through dedicated modules, including NLU extraction, state inference, rule evaluation, and planning. Integration with the OpenAI API is used exclusively for structured extraction and explanation generation, with strict constraints to prevent influence on decision-making. This modular design ensures that each component can be independently developed, tested, and refined, while maintaining a consistent and deterministic execution flow aligned with the system architecture.

The system has been fully implemented as a functional end-to-end prototype, integrating all core components into a unified pipeline and validated through representative interaction scenarios. The design emphasizes deterministic reasoning, transparency, and auditability, making it suitable for real-world rehabilitation support contexts where safety and explainability are critical.

The system design reflects practical considerations for real-world deployment, particularly in safety-sensitive rehabilitation contexts. Its deterministic decision-making, explicit rule traceability, and evidence-grounded explanations align with industry requirements for transparency, reliability, and auditability. In addition, the separation between decision logic and language generation allows the system to be maintained and updated without retraining, making it suitable for environments where clinical protocols evolve over time.

6.2 Core Component Implementation

The core functionality of WellnessBot is implemented through modular components aligned with the system design. The NLU module uses the OpenAI API with a strict schema for structured extraction, followed by deterministic normalization to ensure consistent inputs. The

Rule Engine performs two-pass evaluation with dominance ordering to produce a safe, deterministic action, while the planner selects an appropriate exercise based on phase constraints, exercise priority, equipment, and history.

The RAG module retrieves evidence using a structured query and passes it to the LLM for explanation generation. The logging component records NLU outputs, decisions, rules fired, and audit traces, ensuring full traceability.

This design keeps each component focused, testable, and aligned with its role, supporting a deterministic and auditable system.

6.2.1 NLU Implementation

The NLU module is implemented using the OpenAI API with a strict JSON schema and controlled system prompt, ensuring that all outputs conform to predefined fields. The prompt explicitly restricts the LLM to perform structured extraction only, without making decisions or generating responses.

```
You are a structured NLU extraction component.

You MUST output a single JSON object ONLY.
Do NOT make decisions or suggest exercises.

Use expected_slot to interpret short answers.

Examples:
expected_slot = pain_score
User: "0" → pain_score = 0
User: "no pain" → pain_score = 0

expected_slot = symptom_screen
User: "fever" → symptom_flags = ["fever"], red_flag_terms = ["fever"]
User: "no fever" → symptom_flags = [], negated_terms = ["fever"]

expected_slot = surgery_date
User: "2026-02-01" → surgery_date = "2026-02-01"

... (a lot more example in actual code)

Field definitions:
- weeks_since_event: float weeks (convert days to weeks if explicitly given).
- surgery_type: one of ["arthroscopic_knee_surgery","acl_reconstruction","tkr","sprain_non_surgical","unknown"].
- requested_exercise_text: normalized short phrase ONLY if the user explicitly mentions an exercise they want to do. Otherwise
return an empty string "".
- Never use "unknown" for requested_exercise_text. Use "" instead.
- pain_score: integer 0–3 if present.
- swelling_score: integer 0–3 if present.
- weight_bearing: one of ["none","partial","full","unknown"].
- symptom_screen_done: optional extraction hint only. The downstream policy layer decides conversation progress.
- symptom_flags: list from ["pain","swelling","fever","excessive_bleeding","none"] based only on the current message.
- red_flag_terms: list of explicitly mentioned red flag terms (non-negated), such as fever, excessive bleeding, wound drainage, pus,
chest pain
- negated_terms: list of explicitly negated terms.
- nlu_source: must be "openai".
- surgery_date: date string in YYYY-MM-DD if explicitly provided.

Important:
- A date-only answer like "2026-02-01" is NOT a symptom answer.
- "none" in symptom screening means no symptoms and should set symptom_screen_done=true and symptom_flags=["none"].
- When expected_slot = symptom_screen, do NOT fill pain_score or swelling_score unless the user explicitly gives those values.
- Normalize synonyms:
  - "swell", "swollen", "puffy" -> "swelling"
```

Extraction is performed through a slot-aware function with a fallback mechanism:

```
nlu = extract_with_fallback(user_text, expected_slot)
```

The fallback mechanism applies deterministic post-processing to validate and normalize LLM outputs before reasoning. For example, invalid or noisy inputs such as "unknown", "n/a", or "none" are standardized or filtered to prevent incorrect interpretation in downstream components. Additional normalization ensures consistency across fields, including maintaining a clear separation between symptom detection and severity fields.

This ensures that all inputs are reliable and suitable for deterministic processing.

6.2.2 Rule Engine Implementation

The Rule Engine is implemented as a set of deterministic functions that evaluate the structured NLU output and produce a controlled action. The knowledge graph is loaded from YAML files at runtime and accessed by rule functions to evaluate phase constraints, red flag policies, and exercise eligibility. For example, red flag conditions are explicitly checked to trigger safety actions:

```
if "fever" in nlu.red_flag_terms:
    return Action.ESCALATE
```

Rule execution follows a two-pass evaluation process. In the first pass, non-recommendation rules are evaluated, where multiple rules may be triggered simultaneously. The final action is then resolved using a predefined dominance ordering, ensuring that higher-priority actions override others. An ESCALATE outcome acts as a hard stop, while a FORBID outcome allows supportive recommendation rules to be appended without changing the final decision.

```
for fn in RULES:
    rr = fn(nlu)
```

If no higher-priority condition is triggered, the second pass evaluates recommendation rules. This demonstrates that decision-making is implemented through explicit rule aggregation and deterministic resolution, ensuring consistent and auditable outcomes.

6.2.3 Planner Implementation

The planner is a deterministic selection module that operates on candidate exercises retrieved from phase-specific definitions in the knowledge graph (YAML). It is invoked only after the rule engine permits a RECOMMEND decision and does not override safety-critical outcomes such as ESCALATE or FORBID, ensuring strict adherence to the system's safety-first hierarchy.

Candidates are retrieved using the inferred *surgery_type* and *phase_id*, guaranteeing that all selections remain within clinically defined rehabilitation stages.

```
candidates = list_exercises_for_phase(nlu.surgery_type, phase_id)
```

The planner first applies equipment compatibility filtering, retaining exercises whose required equipment is available. If no compatible exercise exists, it falls back to the full candidate pool to maintain robustness.

```
pool = [ex for ex in candidates if _equipment_compatible(ex, equipment_available)] or candidates
```

Selection is then driven by priority tiers and user history. The planner determines the current working priority based on past recommendations and selects an unseen exercise within that tier. Once all exercises in a tier are completed, it advances deterministically to the next tier, ensuring structured and reproducible progression.

```
working_priority, seen = _working_priority_and_seen(history, phase_id, pool)
candidates_at_target = _unseen_at_priority(pool, working_priority, seen)
```

For mild pain (pain_score == 1), the planner applies a controlled downgrade by reducing the priority tier or falling back to earlier-phase exercises. This adjustment remains constrained by predefined logic and does not override upstream safety decisions.

```
if pain_score == 1:
    downgrade = _downgrade_priority(...)
```

Final selection is strictly deterministic. After candidate exercises have been filtered based on phase eligibility and user constraints, a fixed ordering is applied using the exercise ID. The first candidate in this ordered list is selected as the final recommendation. This step ensures consistent and repeatable outputs for identical input conditions by eliminating ambiguity in cases where multiple valid candidates exist.

```
best = sorted(candidates_at_target, key=lambda ex: ex.exercise_id)[0]
```

If no valid candidate remains, the planner returns a controlled completion outcome, allowing the system to handle progression explicitly. The output is a structured recommendation containing exercise metadata and supporting evidence for downstream explanation.

Overall, the planner provides a deterministic, state-aware mechanism for progressive and personalized exercise selection, while remaining fully transparent, auditable, and subordinate to rule-based safety constraints.

6.2.4 Retrieval-Augmented Explanation Implementation

The RAG module ensures that all generated rehabilitation guidance is grounded in protocol evidence rather than model priors. It is tightly integrated into the recommendation pipeline, with a clear separation of responsibilities: the planner determines the decision, while the LLM is restricted to explanation generation only under controlled inputs.

The retrieval process begins by computing BM25 relevance scores over candidate documents using the tokenized user query, providing an initial ranking based on textual similarity.

```
scores = bm25.score(query_tokens)
```

To enforce alignment with the system's decision-first architecture, retrieval is constrained by planner-approved chunk identifiers. Only evidence associated with the selected exercise is retained, preventing unrelated or misleading information from being introduced.

```
exact_chunk_ids = set(chunk_ids)

for doc, score in zip(documents, scores):
    if doc.chunk_id not in exact_chunk_ids:
        continue
```

Ranking is further stabilised through deterministic score boosting. Exact chunk matches receive a large score increase to guarantee prioritisation, while additional boosting is applied when the document's tool context matches the normalized tool set, ensuring consistent and context-relevant evidence selection

.

```
boosted_score = score

if doc.chunk_id in exact_chunk_ids:
    boosted_score += 100.0

if normalized_tool_set and _safe_str(doc.tool).lower() in normalized_tool_set:
    boosted_score += 5.0
```

The system returns structured, citation-ready evidence rows containing chunk identifiers, exercise metadata, source references, and relevance scores. This structured output is passed directly to the explanation module, enabling transparent and traceable response generation.

```
ranked.append({
    "chunk_id": doc.chunk_id,
    "exercise_id": doc.exercise_id,
    "exercise_name": doc.exercise_name,
    "source_id": doc.source_id,
    "source_url": doc.source_url,
    "phase_id": doc.phase_id,
    "text": doc.text,
    "score": round(boosted_score, 4),
})
```

The retrieved evidence is then provided to the LLM under strict prompt constraints. The model is limited to verbalization of the selected recommendation and supporting evidence, and is explicitly restricted from introducing new advice, modifying decisions, or generating unsupported content. A fixed output structure and citation format are enforced to ensure consistency, grounding, and auditability.

```
You are generating the final recommendation card for a rehabilitation support application.

You must only verbalize the structured recommendation and supporting evidence provided.
Do not change the recommended exercise.
Do not add new exercises, precautions, repetitions, diagnoses, or advice.
Do not invent evidence.
Do not mention internal logic, planner, rules, confidence, or JSON.
If evidence_rows is empty, omit the References section.
If supportive_care is empty, omit the Self Care section.
Use professional, calm, patient-friendly clinical language.

Required sections when data exists, in this order:
1. Self Care
2. Recommended exercise
3. How To Perform
4. References

Do not add any introductory summary sentence before the first section.
Do not restate clinical rationale such as "Mild symptoms noted".

How To Perform grounding rule:
- Rephrase the content from how_to_perform into clear, natural patient-friendly language.
- Do NOT add new steps, repetitions, or procedural details that are not in the source.
- Do NOT omit or summarise any dosage, repetition, or set information present in the source (e.g. "1 set of 10 reps", "increase to 3
sets of 15 reps"). These must appear in full.
- If how_to_perform is empty, omit the How To Perform section.

For each References bullet:
- Use the source_id and source_link from evidence_rows ONLY.
- Do NOT include the evidence text in the References section.
- Format each bullet as: (SOURCE_ID: SOURCE_LINK)
```

Overall, this approach ensures that explanation generation remains deterministic, evidence-aligned, and fully controlled, reinforcing the system's transparency while preventing hallucination or deviation from the planner's decision.

6.3 System Demonstration

6.3.1 User Interface Overview

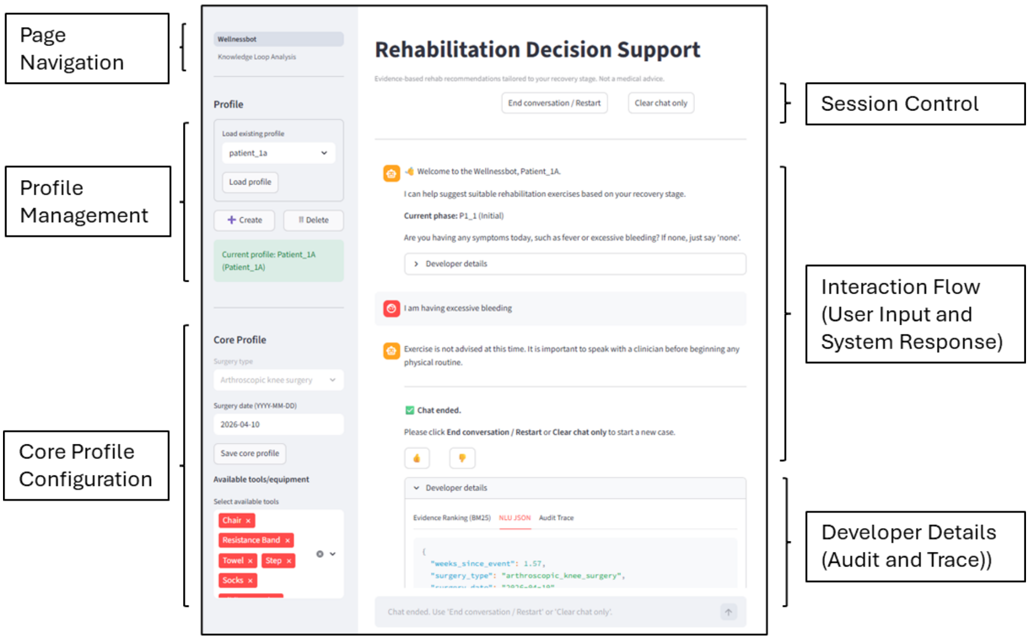


Figure 6: WellnessBot – Rehabilitation Decision Support Interface

The rehabilitation decision interface is implemented as an interactive Streamlit-based application that supports structured user interaction, personalized configuration, and transparent system outputs. The interface is organized into several functional sections, each corresponding to a specific role in the decision-support workflow.

The left panel provides page navigation, allowing users to switch between the main decision interface and the knowledge loop analysis module. Within this panel, the profile management section enables users to load, create, and manage patient profiles, ensuring continuity across sessions. The core profile configuration section captures essential rehabilitation context, including surgery type, surgery date, and available equipment, which are used to determine the recovery phase and constrain feasible exercise recommendations.

The central panel presents the interaction flow, where user inputs and system responses are displayed in a chat-based format. Users report their current condition, and the system generates structured outputs, including recommendations or safety advisories. In the example shown, a report of excessive bleeding triggers a safety-critical response, demonstrating the integration of rule-based decision logic within the interaction flow.

At the top of the interface, session control functions are provided, allowing users to end the conversation, restart, or clear the chat. These controls support interaction management and enable users to initiate new scenarios efficiently.

To ensure transparency, the interface includes a developer details panel that exposes internal system outputs, such as structured NLU results, evidence ranking, and audit traces. This allows inspection of intermediate processing steps and ensures that all system decisions remain traceable and aligned with the underlying deterministic logic.

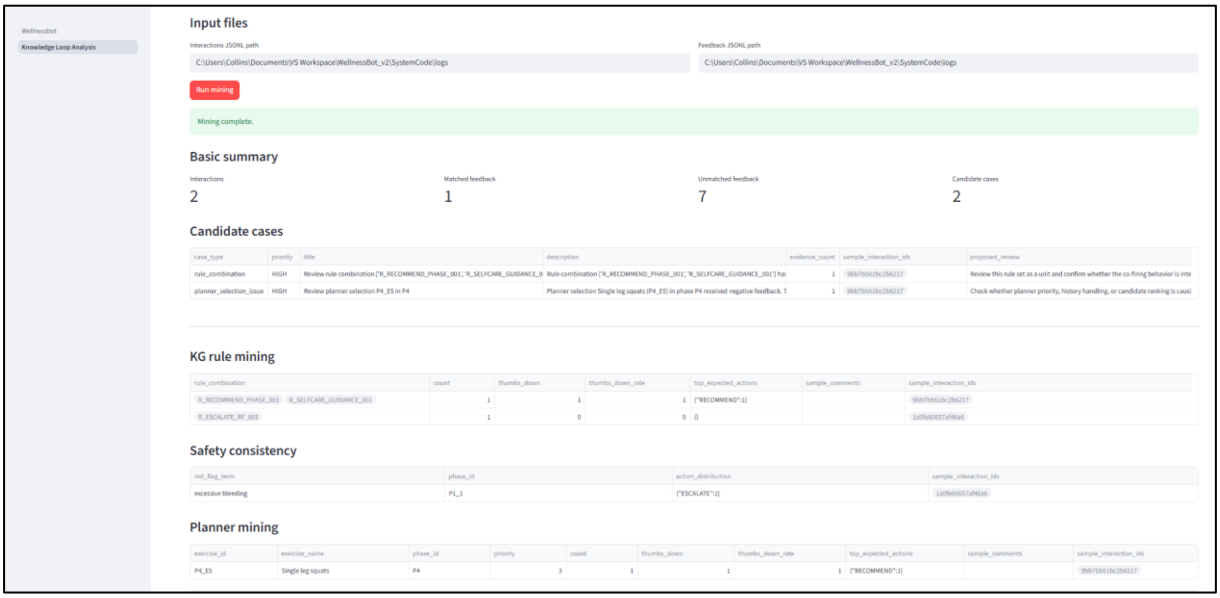


Figure 7: WellnessBot - Knowledge Loop Analysis Interface

The knowledge loop analysis interface provides tools for analysing interaction logs and user feedback to support systematic system evaluation and refinement. It includes a basic summary of system activity, presenting key metrics such as total interactions, matched and unmatched feedback, and the number of candidate cases identified. This overview provides a high-level understanding of system performance and feedback coverage.

The interface surfaces **candidate cases**, which represent interactions flagged by the mining process as potential issues. These include patterns such as rule combinations receiving repeated negative feedback, suboptimal planner selections, or unexpected system outcomes. Each candidate case is accompanied by supporting details, including interaction context, sample interaction identifiers, and proposed review actions, enabling targeted investigation.

To support deeper analysis, the interface includes multiple mining views. The **KG rule mining** view aggregates rule combinations and their associated feedback signals, allowing identification of rules that may require adjustment. The **safety consistency** view evaluates whether safety-critical conditions, such as red-flag symptoms, consistently trigger appropriate actions across interactions. The **planner mining** view analyses exercise selection patterns, highlighting potential issues in priority handling, history management, or candidate ranking.

Together, these components provide a structured and transparent mechanism for identifying system weaknesses and guiding manual refinement of the knowledge graph, rule definitions, and planner logic, without introducing runtime learning.

6.3.2 End-to-End Interaction Flow

The system operates through a complete end-to-end pipeline that transforms user input into a structured, evidence-supported decision. The interaction begins with a guided input collection process, where the system prompts the user for required information such as surgery date, symptom presence, and severity levels. This ensures that all necessary fields are captured before downstream processing.

The collected inputs are processed by the NLU module to produce structured representations, which are then evaluated by the rule engine to determine the appropriate system action. Safety-critical conditions are prioritised, while non-critical cases allow progression to exercise recommendation. In parallel, supportive self-care may be triggered based on symptom severity and retrieved directly from the knowledge graph.

When recommendation is permitted, the planner selects an appropriate exercise based on rehabilitation phase, constraints, and user context. Supporting evidence is then retrieved using a constrained retrieval mechanism, and the LLM generates a natural language explanation based solely on structured inputs and retrieved content. The final output, including exercise guidance, self-care recommendations, and references, is presented to the user, while all intermediate steps are recorded for auditability.

Table 3 summarises the interaction flow for a representative mild-condition scenario.

Step	System Component	Description	Example
1	**User Interaction (Guided Input)**	Collect structured inputs via system prompts.	**Bot:** “Enter surgery date” **User:** 2026-04-10 **Bot:** “Are you having any symptoms?” **User:** “none” **Bot:** “Pain level (0–3)?” **User:** “Pain 1” **Bot:** “Swelling level (0–3)?” **User:** “Swelling 1”
2	**NLU Extraction**	Convert input into structured fields.	**Extracted Fields:** • surgery_date: 2026-04-10 • pain_score: 1 • swelling_level: 1 • symptom_flags: “none” • red_flag_terms: [] • negated_terms: []
3	**Rule Engine Evaluation**	Evaluate safety rules and determine action.	**Rule Engine Result:** • No red-flag symptoms detected • Mild condition identified • Action: Proceed conservatively • Trigger: supportive self-care
4	**Knowledge Graph Retrieval (Self-Care Actions)**	Retrieve self-care actions from KG.	**Self-Care (from KG):** • Use pain relief as prescribed • Ice and elevate for 10 minutes (when needed and after exercise)
5	**Planner Selection**	Select exercise based on phase and constraints.	**Planner Output:** • Rehabilitation phase: P2 (Early) • Pain level mild → downgrade intensity • Selected exercise: Straight leg raises (supine)
6	**RAG Retrieval**	Retrieve supporting evidence using chunk filters.	**Retrieved Evidence:** • S2: Knee Arthroscopy Rehabilitation Protocol
7	**LLM Explanation (Constrained)**	Generate explanation from structured inputs and evidence.	**How to Perform:** Lying flat, keep your knee straight and lift your whole leg about 30 cm off the bed. Hold for 3 seconds, then relax. Repeat 10–20 times, 2–3 times daily.
8	**Final Output**	Present recommendation, self-care, and references.	**Output:** • SELF CARE • RECOMMENDED EXERCISE • HOW TO PERFORM • REFERENCES
9	**Audit Trace Logging**	Record all intermediate outputs for traceability.	**Logged Data:** NLU output, rule result, self-care trigger, planner output, retrieved evidence, final response

Table 3: End-to-End Decision Pipeline of WellnessBot

This structured flow demonstrates how the system integrates guided input collection, deterministic rule-based reasoning, planner-driven selection, and evidence-grounded explanation within a transparent and auditable pipeline.

6.3.3 Decision Transparency and Audit Trace

The system provides full decision transparency through a structured audit trace that captures all intermediate processing steps for each interaction. This ensures that every system output can be traced back to its underlying inputs, rule evaluations, and evidence sources.

For each interaction, the system records the structured output generated by the NLU module, including fields such as surgery type, weeks since event, pain score, swelling score, and detected symptom indicators. These structured inputs form the basis for deterministic reasoning and are stored as part of the interaction log.

The rule engine evaluation is explicitly captured through a list of triggered rules, including their identifiers, associated actions, and rationales. For example, rules such as *R_SUPPORTIVE_NONBLOCK_RF_006*, *R_SELFCARE_GUIDANCE_001*, and *R_RECOMMEND_PHASE_001* are logged together with their respective contributions to the final decision. This enables clear tracing of how multiple rules combine to produce the final action.

When a recommendation is made, the planner output is recorded in detail, including the selected exercise, rehabilitation phase, priority level, and associated constraints. The log also captures supporting metadata such as stop conditions, equipment requirements, and self-care routines derived from the knowledge graph. This ensures that the reasoning behind exercise selection is fully transparent.

The retrieval process is similarly traceable, with evidence chunks explicitly recorded using chunk identifiers and source references. These chunk identifiers are passed to the retrieval module to constrain evidence selection, ensuring that the explanation remains aligned with the planner's decision.

All audit information is accessible through the developer interface, where structured logs expose components such as NLU outputs, rule evaluation results, planner decisions, and retrieved evidence. This allows developers and domain experts to inspect system behaviour at each stage, verify correctness, and identify potential issues.

Overall, the audit trace reinforces the system's deterministic and neuro-symbolic design by ensuring that all decisions are explainable, reproducible, and fully traceable, which is essential for deployment in safety-critical rehabilitation contexts.

6.3.4 Example Scenarios

To demonstrate system behaviour across different conditions, representative scenarios are presented to illustrate how the system responds to safety-critical inputs, normal rehabilitation cases, and incomplete information. These scenarios focus on the resulting system actions rather than internal processing, which has been detailed in Section 7.3.2.

i. **Scenario 1: Normal Rehabilitation Condition (RECOMMEND)**

- **Input:** User reports no symptoms, with pain score of 0 and swelling score of 2 during Phase P2.
- **System Behaviour:** The system detects no safety concerns and allows progression to recommendation. Based on the rehabilitation phase and symptom levels, the planner selects an appropriate exercise. Supportive self-care is also provided.
- **Outcome:** RECOMMEND — exercise is provided along with self-care guidance (ice for 20 minutes when needed or after exercise).

ii. **Scenario 2: Safety-Critical Condition (FORBID)**

- **Input:** User reports having a high fever during Phase P1-1.
- **System Behaviour:** The system identifies fever as a safety-critical condition and restricts any exercise recommendation. Instead, it provides conservative guidance and supportive self-care.
- **Outcome:** FORBID — user is advised not to proceed with exercise and to follow prescribed medication and cooling measures.

iii. **Scenario 3: Red-Flag Condition (ESCALATE)**

- **Input:** User reports "I have noticed quite a bit of bleeding from the incision" during Phase P2.

- **System Behaviour:** The system identifies excessive bleeding as a red-flag condition and immediately prioritises safety. Exercise recommendation is bypassed, and escalation is triggered.
- **Outcome:** ESCALATE — user is advised to seek immediate medical attention.

These scenarios demonstrate that the system consistently enforces safety constraints, provides appropriate recommendations under normal conditions, and handles incomplete inputs through controlled clarification, ensuring reliable and deterministic behaviour across different interaction contexts.

6.4 Implementation Challenges and Resolutions

During implementation, several technical challenges were encountered in ensuring reliable structured input, enforcing safety constraints, maintaining deterministic planning behaviour, and controlling retrieval quality. These were addressed through targeted design decisions aligned with the system's neuro-symbolic architecture.

i. **Challenge 1: Reliable Structured Input from Natural Language**

User inputs are often incomplete or ambiguous, which can affect downstream decision-making. Extracting consistent values for pain and swelling levels from free-form text proved unreliable.

Resolution: A guided input mechanism was introduced using predefined options (e.g., 0–3 scales for pain and swelling), complemented by schema-constrained NLU extraction. This ensures consistent and valid inputs while reducing ambiguity in critical fields.

ii. **Challenge 2: Handling Phase-Specific Exceptions in Knowledge Graph**

The rehabilitation protocol includes exceptions where standard rules do not uniformly apply across phases. For example, Phase P1_1 requires stricter handling of certain conditions, such as fever, compared to later phases.

Resolution: Explicit exception handling was incorporated into the knowledge graph and rule definitions, allowing phase-specific policies to override general behaviour. This ensures that safety and clinical intent are preserved across different recovery stages.

iii. **Challenge 3: Deterministic and Context-Aware Planner Behaviour**

The planner must balance multiple factors, including rehabilitation phase, user condition, and exercise history, while maintaining deterministic selection.

Resolution: A structured selection strategy was implemented, incorporating phase filtering, priority tiers, and condition-based adjustments (e.g., pain-based intensity downgrade). This ensures consistent and context-aware exercise selection without introducing randomness.

iv. **Challenge 4: Controlling Retrieval Quality in RAG**

Ensuring that retrieved evidence remains aligned with clinically relevant content was a key challenge. Retrieval based solely on lexical similarity could surface text that is related in wording but not directly linked to the exercise selected by the planner.

Resolution: Retrieval was constrained to planner-approved chunk identifiers, ensuring that only evidence associated with the selected exercise is considered. Ranking is then performed only within this constrained set, significantly reducing unsupported responses and improving traceability.

v. **Challenge 5: Balancing Relevance and Deterministic Behaviour in Retrieval**

Maintaining both relevance and deterministic behaviour in retrieval posed an additional challenge. BM25 scoring alone can be sensitive to variations in wording and may not consistently prioritize operationally preferred evidence.

Resolution: A hybrid scoring approach was implemented by combining BM25 relevance with deterministic score adjustments, including boosts for exact chunk matches and tool alignment, followed by stable sorting. This ensures that retrieval results remain both relevant and predictable.

6.5 Logging and Auditability

The system implements a structured logging mechanism to ensure full traceability and auditability of all interactions. Each user interaction is recorded as a structured log entry, capturing both input data and all intermediate processing outputs across the decision pipeline.

For every interaction, the system logs the structured output generated by the NLU module, including fields such as surgery type, weeks since event, pain score, swelling level, and symptom indicators. This structured representation serves as the foundation for downstream reasoning and ensures that all inputs are explicitly recorded.

The decision-making process is fully traceable through recorded rule evaluations. The system logs the set of rules triggered for each interaction, including their identifiers, actions, rationales, and confidence contributions. This allows developers and domain experts to understand how multiple rules contribute to the final system action.

When a recommendation is generated, the planner output is captured in detail, including the selected exercise, rehabilitation phase, priority level, and associated constraints. Additional metadata, such as stop conditions and self-care routines retrieved from the knowledge graph, are also recorded, ensuring that the reasoning behind each recommendation is transparent.

The retrieval process is similarly auditable, with evidence chunks recorded using unique identifiers and source references. These identifiers ensure that all explanations presented to the user can be traced back to specific protocol sources, supporting verification and consistency.

All logs are stored in a structured format and are accessible through the developer interface, where intermediate states such as NLU outputs, rule evaluations, planner decisions, and retrieved evidence can be inspected. This enables systematic debugging, validation, and performance analysis.

Importantly, the logging mechanism operates independently of the decision pipeline and does not influence runtime behaviour. Instead, it functions as an observation layer, ensuring that all system outputs remain reproducible, explainable, and verifiable. This level of auditability is critical for deployment in safety-sensitive rehabilitation contexts.

6.6 Knowledge Loop and Mining

To support continuous system improvement, an offline knowledge loop is implemented to analyse interaction logs and user feedback. The objective of this module is to systematically identify weaknesses in rule behaviour, planner decisions, and safety handling, while preserving the deterministic nature of the runtime system. Rather than introducing automated learning, the system applies structured mining techniques to generate candidate cases for manual refinement of the knowledge graph and rule definitions.

i. **KG Rule Mining**

- **Objective**: Identify rule combinations that consistently lead to negative feedback.
- **Method**: Interaction logs are grouped by triggered rule sets (rule_ids). For each combination, the system computes occurrence count and negative feedback rate, forming aggregates such as *rule_combination* → count, *thumbs_down_rate*.
- **Interpretation**: Frequently occurring rule combinations with high negative feedback indicate potential issues in rule design or interaction effects.
- **Outcome**: Candidate cases are generated to guide review and refinement of rule definitions.

ii. **Safety Consistency Analysis**

- **Objective:** Verify that safety-critical conditions consistently trigger appropriate actions.
- **Method:** Logs are filtered based on detected red-flag terms (e.g., "fever"), and analysed by rehabilitation phase and resulting action distribution, forming mappings such as red_flag_term → phase_id → action_distribution.
- **Interpretation:** Red-flag conditions should consistently result in **ESCALATE**. Any deviation indicates a potential safety inconsistency.
- **Outcome:** Identify violations of safety policies and ensure consistent rule behaviour across scenarios.

iii. **Planner Mining**

- **Objective:** Evaluate the quality and suitability of planner-selected exercises.
- **Method:** Planner outputs are grouped by exercise and rehabilitation phase. For each grouping, selection frequency and negative feedback rate are computed, forming aggregates such as exercise_id → phase_id → count, thumbs_down_rate.
- **Interpretation:** Exercises with high negative feedback may indicate issues in prioritization, suitability, or constraint handling.

- **Outcome:** Candidate cases are generated to guide refinement of planner selection logic.

The outputs from these mining processes are consolidated into candidate cases, which provide actionable insights for system improvement. All updates are performed manually, ensuring that the system remains deterministic, transparent, and auditable while continuously improving based on real interaction data.

7 Results and Progress

7.1 Evaluation Overview

The performance of the proposed system is evaluated with a focus on decision correctness and safety in rehabilitation support. The evaluation is conducted on a set of 33 test scenarios, of which 32 cases are used for metric computation. One scenario corresponds to the final rehabilitation stage (P5 – Return to Sport), where no decision action is required. Instead, the system provides a completion message indicating recovery progress and guidance for resuming normal physical activity. As this case does not involve a decision outcome (RECOMMEND, FORBID, or ESCALATE), it is excluded from quantitative evaluation.

The evaluation follows a two-layer framework. The primary layer assesses decision-level performance, focusing on the correctness of the system's action selection (RECOMMEND, FORBID, ESCALATE), which directly reflects clinical safety. The secondary layer evaluates the appropriateness of exercise and self-care recommendations generated by the planner, conditional on correct decision outcomes.

In addition, a baseline comparison is conducted using an LLM-only approach to highlight the limitations of purely generative decision-making. This enables a direct comparison between the proposed deterministic neuro-symbolic system and an unconstrained LLM baseline, providing insight into the reliability, consistency, and safety of both approaches.

7.2 Evaluation Framework

The system is evaluated using a two-layer framework designed to assess both decision correctness and recommendation quality. This structure reflects the modular architecture of the system, where safety-critical decisions are handled independently from exercise selection. By separating these components, the evaluation ensures that decision reliability is assessed prior to recommendation validity.

7.2.1 Layer 1 – Decision -Level Evaluation

The primary layer evaluates whether the system selects the correct action, defined as one of RECOMMEND, FORBID, or ESCALATE. This layer directly reflects the safety and correctness of the system's decision-making process and is therefore considered the most critical component of the evaluation.

Performance is assessed using a confusion matrix, comparing predicted actions against ground truth labels across all valid scenarios. From this, standard classification metrics are computed:

$$\text{Accuracy} = \frac{\text{Total number of predictions}}{\text{Number of correct predictions}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} \quad = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1-score} = \frac{2 \text{ x Precision x Recall}}{\text{Precision} + \text{Recall}}$$

Accuracy is computed as an overall measure across all scenarios, while precision, recall, and F1-score are evaluated for each decision class. Recall is emphasized for safety-critical actions, as it reflects the system's ability to correctly identify all cases requiring escalation or restriction. A lower recall in these cases indicates potential safety risks due to missed critical conditions.

Representative scenarios are presented to illustrate both correct predictions and failure cases, including instances of missed escalation or overly restrictive forbidding. These examples provide qualitative insight into system behaviour and complement the quantitative evaluation results.

7.2.2 Layer 2 – Exercise -Level Evaluation

The second layer evaluates the appropriateness of the recommended exercise and self-care actions, conditional on correct decision outcomes. This layer serves as a secondary validation step, ensuring that recommendations generated by the planner are aligned with the user's rehabilitation phase, symptom condition, and operational constraints.

The planner produces a single exercise recommendation using a deterministic selection strategy guided by phase eligibility, exercise priority, equipment availability, and user history. While multiple exercises may be clinically valid, only one is selected, and evaluation is therefore based on whether the chosen recommendation is appropriate within the given context.

Sample scenarios are used to assess both correct and incorrect recommendations, particularly in cases where multiple valid alternatives exist. Exercise and self-care outputs are evaluated consistently under the same criteria, ensuring alignment with protocol-defined rehabilitation guidance.

7.3 Baseline Comparison with LLM

To evaluate the effectiveness of the proposed neuro-symbolic approach, a baseline comparison is conducted using an LLM-based decision model. This baseline represents a generative decision-making approach, where the LLM is responsible for producing the final decision label without the use of a knowledge graph, rule engine, or deterministic planning mechanism.

7.3.1 Baseline Setup

The LLM baseline is evaluated using the **same set of scenarios** described in the evaluation dataset, ensuring a fair and consistent comparison. For each scenario, the model is prompted to output a single decision label (RECOMMEND, FORBID, or ESCALATE) based on predefined rules embedded within the prompt.

While the model primarily operates on natural language input, additional structured context is explicitly provided within the prompt, including the reference date, weeks since surgery, and the computed rehabilitation phase. The model is instructed to use the given phase directly without recalculating it, ensuring consistency across all scenarios. Despite the availability of this structured information, the LLM remains responsible for performing the final decision-making based on the provided inputs.

It is important to note that the same LLM model is also used in the proposed system, but only for structured NLU extraction. In contrast, the baseline uses the LLM for end-to-end decision-making. This design enables a controlled comparison between two roles of the same model: generative decision-making versus structured extraction followed by deterministic reasoning. As such, differences in performance can be attributed to the system architecture rather than model capability.

7.3.2 Baseline Performance Overview

The LLM baseline achieves an overall accuracy of **90.62%** across the evaluated scenarios. Performance varies across decision classes. The model demonstrates strong performance in RECOMMEND cases, achieving high precision and recall, indicating reliable behaviour in routine scenarios. Performance in ESCALATE cases shows relatively high recall, suggesting that most critical conditions are identified. However, precision is lower, indicating a tendency to over-trigger escalation in ambiguous situations.

In contrast, performance in FORBID cases is less consistent, with lower recall observed. This indicates that the model does not always correctly enforce restriction decisions when required, leading to potential safety gaps. Overall, while the baseline performs well in general scenarios, inconsistencies remain in safety-critical decision boundaries.

7.3.3 Limitation of LLM-Based Decision-Making

The observed results reveal inherent limitations of LLM-based decision-making in safety-critical applications. While the model can follow general patterns and rules when explicitly provided, it exhibits inconsistent behaviour when distinguishing between moderate and high-risk conditions. This is particularly evident in cases requiring nuanced interpretation of symptoms to differentiate between FORBID and ESCALATE decisions.

Importantly, these limitations persist even when structured contextual inputs such as rehabilitation phase and weeks since surgery are provided. This indicates that the primary challenge lies not in the availability of information, but in the model's ability to reliably enforce decision logic.

These limitations stem from the generative nature of LLMs, which lack explicit constraint enforcement and deterministic reasoning. As a result, decision outcomes may vary depending on phrasing, context, or implicit model biases, leading to reduced reliability and lack of auditability.

In contrast, the proposed system separates language understanding from decision-making, using the LLM only for structured NLU extraction while enforcing decisions through a deterministic rule engine and knowledge graph. This ensures consistent application of safety constraints, eliminates ambiguity in critical decision pathways, and provides transparent, traceable outcomes. These findings highlight that the effectiveness of LLM-based systems depends not only on model capability, but more importantly on how the model is integrated within the overall system architecture.

7.4 Results and observations

7.4.1 End-to-End System Performance

The performance of the proposed system is evaluated in an end-to-end setting, encompassing the full pipeline from NLU extraction to deterministic reasoning through the knowledge graph and rule engine. The resulting decision-level confusion matrix is presented in Table 4.

	Predict: **RECOMMEND**	Predict: **FORBID**	Predict: **ESCALATE**
Actual: RECOMMEND	16	0	0
Actual: FORBID	0	7	0
Actual: ESCALATE	0	2	7

Table 4: Decision-Level Confusion Matrix for End-to-End System (LLM + KG + Rule Engine)

The corresponding classification performance metrics derived from the confusion matrix are summarized in Table 5.

Label	**Precision**	**Recall**	**F1 Score**
RECOMMEND	1	1	1
FORBID	0.7778	1	0.875
ESCALATE	1	0.7778	0.875
Accuracy	0.9375		

Table 5: Classification Performance Metrics Derived from Confusion Matrix

The system achieves an overall accuracy of 0.9375, demonstrating strong performance across all decision categories. RECOMMEND cases are classified with perfect precision and recall, indicating consistent handling of routine, low-risk scenarios. FORBID cases also achieve perfect recall, ensuring that all required restrictions are correctly enforced.

For ESCALATE cases, the system achieves a recall of 0.7778, with two instances misclassified as FORBID. This represents the only observed failure pattern in the evaluation. Importantly, no cases are incorrectly classified as RECOMMEND when escalation is required, indicating that the system avoids unsafe recommendations and maintains a conservative decision bias.

The observed misclassifications are attributed to upstream NLU extraction rather than the deterministic reasoning components. In these cases, critical symptoms are not correctly extracted or mapped to structured red-flag indicators during NLU processing. As a result, the rule engine does not receive the necessary signals to trigger escalation, leading to a downstream FORBID decision instead.

This behaviour highlights an important characteristic of the system: while the rule engine enforces decision logic deterministically and consistently, the overall system performance remains dependent on the quality of NLU extraction. Despite this dependency, the system maintains high reliability, with errors confined to a small number of borderline cases.

Overall, the results demonstrate that deterministic rule-based reasoning ensures consistent enforcement of decision boundaries, while the primary limitation lies in upstream language understanding. This confirms the effectiveness of separating language understanding from decision-making, where improvements in NLU directly translate to improved end-to-end system performance without requiring changes to the decision logic.

7.4.2 Exercise-Level Performance Analysis

The exercise-level evaluation assesses the appropriateness of the planner's recommended exercises, conditional on correct decision outcomes. Representative scenarios are presented in Table 6.

No	Profile ID	Equipment	Test Phase	Pain Score	Swelling Score	Actual Exercise	Predicted Exercise	Remark
1	Patient A	NA	P1-1	pain 0	swelling 1	P1_1_E1	P1_1_E1	
2	Patient A	NA	P1-1	1	2	P1_1_E1	P1_1_E1	Downgrade
3	Patient A	Chair	P1-2	1	2	P1_1_E3	P1_1_E3	Downgrade
4	Patient A	Chair	P1-2	0	0	P1_2_E2	P1_2_E2	
5	Patient B	NA	P2	pain 0	swelling 1	P2_E1	P2_E1	
6	Patient B	Stationary Bicycle	P2	0	2	P2_E2	P2_E2	
7	Patient B	Stationary Bicycle	P2	0	0	P2_E3	P2_E3	
8	Patient C	NA	P2	pain 0	swelling 2	P2_E1	P2_E1	
9	Patient C	NA	P3	0	0	P2_E1/E2	P3_E1	
10	Patient C	Towel	P3	1	0	P2_E1/E2	P2_E1	Downgrade
11	Patient C	Towel	P3	0	1	P3_E2	P3_E2	
12	Patient D	NA	P4	0	0	P4_E3	P4_E3	
13	Patient D	Stool	P4	0	2	P4_E4	P4_E4	
14	Patient G	Socks	P1-2	1	0	P1_1_E2	P1_1_E2	Downgrade

15	Patient H	Thick carpet	P3	pain 1	swelling 2	P2_E4	P2_E4	Downgrade
16	Patient I	Resistance band	P4	pain 1	1	P3_E5/E6	P3_E5	Downgrade

Table 6: Representative Exercise-Level Evaluation Scenarios

Out of 16 evaluated scenarios, 10 cases are exact matches with the expected exercise based on their priority tiers, while the remaining 6 cases are classified as successful downgrades due to pain conditions. In this evaluation, both exact matches and clinically valid downgrades are considered correct outcomes, as they satisfy phase constraints and safety requirements. As a result, the planner achieves an effective accuracy of 100% under the defined evaluation criteria, with no incorrect or unsafe recommendations observed.

Downgrade cases occur when the planner selects an alternative exercise that differs from the predefined expected label but remains clinically valid within the same rehabilitation phase. This reflects the inherent flexibility in rehabilitation planning, where multiple exercises may be appropriate for a given phase and condition.

These results indicate that the planner consistently generates valid and safe recommendations within the appropriate rehabilitation phase. The absence of incorrect recommendations demonstrates that constraint-based filtering and deterministic selection effectively prevent unsafe or phase-inappropriate outputs.

Overall, the evaluation confirms that the planner achieves high reliability in exercise recommendation, with variations limited to acceptable alternatives rather than true errors. This supports the design choice of selecting a single exercise from a set of valid candidates while maintaining clinical safety and practical applicability.

7.4.3 Self-Care Recommendation Evaluation

The evaluation of self-care recommendations assesses whether the system provides appropriate guidance aligned with the user's rehabilitation phase and condition. To ensure a valid assessment, self-care evaluation is conducted only on cases where the primary decision outcome is correctly classified. Cases with incorrect decision outcomes and escalation scenarios are excluded, as self-care recommendations are not applicable under these conditions.

Based on this filtering, a total of 22 valid scenarios are evaluated. Representative examples are presented in Table 7, while the full evaluation results are provided in Appendix E.

No.	Phase	Expected Self-Care	Predicted Self-Care	Result
1	P1-1	Prescribed medicine + Ice 20 mins hourly	Prescribed medicine + Ice 20 mins hourly	Pass
2	P1-2	Prescribed pain relief + Ice 20 mins 3-4times per day	Prescribed pain relief + Ice 20 mins 3-4times per day	Pass
3	P2	Ice 10 mins when needed / after exercise	Ice 10 mins when needed / after exercise	Pass
4	P3	Hot towel 10 mins	Hot towel 10 mins	Pass
5	P4	Walk 10 minutes before exercise.	Walk 10 minutes before exercise.	Pass

Table 7: Representative Self Care Evaluation Scenarios

Across all valid cases, the predicted self-care recommendations match the expected outputs exactly, resulting in 100% accuracy. No incorrect or downgrade cases are observed.

This consistency reflects the deterministic nature of the rule-based design, where self-care actions are directly derived from structured knowledge representations defined in the protocol. Unlike exercise recommendations, which may involve multiple clinically valid alternatives, self-care guidance is more standardized and therefore less prone to variation.

The results indicate that the system reliably generates safe and protocol-aligned self-care recommendations across all evaluated conditions. The absence of incorrect outputs further demonstrates that the system effectively enforces phase-specific constraints and avoids unsafe or inappropriate guidance.

Overall, the evaluation confirms that the self-care component of the system is highly reliable when conditioned on correct decision outcomes. This highlights the importance of accurate upstream decision-making, as improvements in decision and NLU performance will directly enhance the validity of downstream self-care recommendations.

7.4.4 Comparative Analysis

A comparative evaluation is conducted between the LLM-based baseline and the proposed neuro-symbolic system to assess differences in decision reliability and safety. Both approaches utilize the same underlying LLM model; however, they differ fundamentally in how the model is integrated within the system. In the baseline, the LLM performs end-to-end decision-making, whereas in WellnessBot, the LLM is restricted to structured NLU extraction, with all decisions governed by deterministic reasoning through the knowledge graph and rule engine.

The performance of the LLM baseline is summarized in Table 8, while the corresponding results for WellnessBot are presented earlier in Table 5**.**

Label	Precision	Recall	F1 Score
RECOMMEND	0.9375	1.0000	0.9677
FORBID	0.8889	0.8000	0.8421
ESCALATE	0.8571	0.8571	0.8571
Accuracy	0.9062		

Table 8: Performance Metrics for LLM Baseline

A direct comparison between the LLM baseline and WellnessBot is summarized in Table 9.

Metric	**LLM Baseline**	**WellnessBot**
Accuracy	0.9062	0.9375
RECOMMEND Recall	1.0000	1.0000
FORBID Recall	0.8000	1.0000
ESCALATE Recall	0.8571	0.7778
Unsafe Recommendation (FORBID → RECOMMEND)	Yes	No
Decision Consistency	Moderate	High
Deterministic Reasoning	No	Yes

Table 9: Comparison of Decision-Level Performance Between LLM Baseline and WellnessBot

At the decision level, WellnessBot achieves higher overall accuracy (0.9375) compared to the LLM baseline (0.9062). While both approaches demonstrate perfect recall in RECOMMEND cases, clear differences emerge in safety-critical decisions.

For FORBID cases, WellnessBot achieves perfect recall (1.0), ensuring that all required restrictions are correctly enforced. In contrast, the LLM baseline achieves a recall of 0.8, indicating that a portion of restriction cases are not correctly identified. Notably, this includes instances where FORBID cases are incorrectly predicted as RECOMMEND, representing potentially unsafe recommendations. Such unsafe behaviour is not observed in WellnessBot.

For ESCALATE cases, the LLM baseline achieves a higher recall (0.8571) compared to WellnessBot (0.7778). However, this comes at the cost of reduced decision consistency, as the baseline exhibits confusion between ESCALATE and FORBID decisions. In contrast, misclassifications in WellnessBot are limited and can be attributed to upstream NLU extraction errors rather than failures in the deterministic reasoning process.

A key distinction lies in errors. In WellnessBot, errors occur only when critical symptoms are not correctly extracted or mapped during NLU processing. When accurate structured inputs are provided, the rule engine consistently enforces correct decision logic. In contrast, the LLM baseline exhibits inconsistent decision behaviour despite being provided with structured contextual inputs such as rehabilitation phase, indicating limitations in reliably applying decision rules.

At the recommendation level, WellnessBot further demonstrates stronger control through constraint-based reasoning. Exercise recommendations are consistently valid within the appropriate rehabilitation phase, with variations limited to downgrade cases that remain clinically acceptable. For self-care recommendations, the system achieves 100% accuracy across all valid scenarios, reflecting the deterministic mapping from structured knowledge to output actions.

In contrast, the LLM baseline lacks explicit constraint enforcement, leading to variability in decision outcomes, particularly in boundary and safety-critical conditions. While the baseline performs adequately in routine scenarios, it does not provide the level of reliability required for consistent clinical decision support.

Overall, the results demonstrate that WellnessBot provides more reliable, consistent, and auditable decision-making compared to the LLM baseline. By separating language understanding from decision logic, the system ensures systematic enforcement of safety constraints while maintaining transparency and traceability. These findings highlight that system architecture, rather than model capability alone, plays a critical role in achieving robust and safe outcomes in clinical decision-support systems.

7.5 Academic and Market Relevance

This work contributes academically by demonstrating a practical implementation of a neuro-symbolic decision-support system, where language understanding and reasoning are explicitly separated. Unlike purely data-driven approaches, the system integrates structured knowledge representation with deterministic rule-based reasoning, enabling transparent, explainable, and auditable decision-making. The evaluation further highlights the limitations of LLM-based decision-making and reinforces the importance of system architecture in safety-critical applications.

From a market perspective, the system addresses a real-world gap in post-discharge rehabilitation support, where patients often lack continuous clinical guidance. By providing structured recommendations with clear reasoning, WellnessBot has potential applications in digital health platforms, remote patient monitoring, and rehabilitation support systems. Its deterministic and auditable design also aligns with regulatory expectations in healthcare settings, where reliability and traceability are critical.

7.6 Deviation from initial plan

Several deviations from the initial plan were introduced during development to improve system robustness, consistency, and scalability.

First, **the introduction of a user profile component** was not part of the original design, as it introduced additional implementation complexity. However, it was later incorporated to support phase inference, personalization, and consistent state tracking across interactions. This allows the system to determine rehabilitation phase based on surgery date and ensures that recommendations are context-aware and aligned with the user's recovery progression.

Second**, the chat interaction design was revised to incorporate structured input controls for selected variables**, specifically pain and swelling levels. This replaced free-text input for these variables, reducing ambiguity and improving consistency in NLU extraction. By constraining only these key inputs, the system minimizes variability in interpretation while retaining flexibility for other user-provided information, ensuring more reliable downstream reasoning.

Third**, the RAG component was modified from a one-to-one retrieval approach to a BM25-based ranking mechanism**. Although the current implementation operates on a small number of candidate evidence chunks (often one to two), BM25 is applied to maintain a consistent and systematic retrieval framework. This avoids direct mapping between queries and evidence, improving robustness to variations in input phrasing while preserving deterministic behaviour through constrained candidate filtering.

The adoption of BM25 also prepares the system for future expansion, where a larger knowledge base with multiple candidate evidence chunks per query is expected. In such scenarios, ranking becomes essential for selecting the most relevant and contextually appropriate evidence. By introducing BM25 at this stage, the system avoids the need for architectural changes when scaling, ensuring continuity in design and evaluation.

These deviations reflect iterative refinement based on implementation insights, ensuring that the system remains practical, consistent, and aligned with its design principles of determinism, auditability, and scalability.

8 Challenges and Roadblocks

While the system demonstrates a structured and explainable decision-support pipeline, several challenges remain that reflect the complexity of translating clinical knowledge into a deterministic and scalable framework. These challenges arise from inherent design trade-offs between coverage, personalisation, and knowledge consistency within a controlled system scope, and are accompanied by clear pathways for incremental improvement.

The **limited protocol coverage** represents a key roadblock, as the system currently supports only a single rehabilitation protocol focused on post-arthroscopic knee surgery, restricting its applicability to broader clinical contexts. This constraint is intentional to ensure depth and correctness of knowledge representation; however, it can be progressively addressed through **modular extension of the YAML-based knowledge graph**, where additional protocols are incorporated with validated phase definitions and aligned safety rules, preserving consistency across the system.

The **limited granularity of patient condition** modelling presents another roadblock, as the system relies primarily on phase and basic symptom inputs, resulting in a relatively coarse level of personalisation. In real-world rehabilitation, outcomes are influenced by individual factors such as mobility, recovery rate, and tolerance. This limitation can be mitigated by **extending the user profile schema and introducing more granular rule conditions**, enabling the planner to make more context-aware decisions while maintaining coherence within the deterministic reasoning pipeline.

Finally, **knowledge consistency across multiple clinical sources** remains an ongoing roadblock, as rehabilitation guidelines are often unstructured and vary in scope, requiring manual consolidation into a unified representation. While a primary protocol provides a consistent foundation, supplementary knowledge such as self-care recommendations introduces additional complexity. This can be addressed through **structured curation and validation processes**, supported by consistent schema design and controlled integration of new knowledge sources to ensure long-term maintainability.

Collectively, these roadblocks highlight the inherent challenges of developing a clinically grounded, knowledge-driven system that prioritises safety, explainability, and auditability. Importantly, the system architecture provides a robust foundation for incremental and controlled enhancement, allowing protocol coverage, personalisation depth, and knowledge consistency to be systematically improved without requiring fundamental changes to the underlying design.

9 Future Work

A key direction for future work lies in **improving the scalability of the system** while preserving its deterministic and explainable design. Although the current implementation focuses on a single rehabilitation protocol, the underlying YAML-based knowledge graph is inherently modular, enabling additional protocols to be incorporated without altering the core reasoning pipeline. Scaling the system will require structured knowledge ingestion processes, including standardised schema definitions, validation of clinical consistency, and controlled integration of new protocols and exercises. As the knowledge base expands, optimisation of rule evaluation and planner selection will also be necessary to ensure efficiency and maintain system responsiveness.

In addition, future enhancements will focus on **improving the personalisation of recommendations**. The current system relies primarily on rehabilitation phase and basic symptom inputs, which limits its ability to reflect individual patient variability. Incorporating richer user attributes, such as mobility level, recovery progression, and exercise tolerance, would enable more context-aware decision-making. This can be achieved by extending the user profile schema and introducing additional rule conditions, while maintaining consistency and interpretability within the deterministic reasoning framework.

Finally, future work will explore more **comprehensive evaluation strateg**ies to strengthen confidence in system performance. This includes expanding scenario-based testing to cover a wider range of clinical conditions and edge cases, as well as comparing system outputs with expert clinical judgement. Such evaluation efforts will provide stronger evidence of system reliability and support the transition from a prototype system to a more clinically relevant decision-support tool.

10 Conclusion

This project presented the design and implementation of WellnessBot, a neuro-symbolic decision-support system for knee rehabilitation that prioritises deterministic reasoning, explainability, and clinical safety. By integrating a structured YAML-based knowledge graph with a rule-based engine and a constraint-aware planner, the system demonstrates a decision-first architecture in which recommendations are derived from explicitly defined knowledge rather than learned implicitly from data. The use of LLMs is deliberately constrained to structured information extraction and explanation generation, ensuring that all decisions remain transparent, auditable, and grounded in validated clinical sources.

The system illustrates how rehabilitation knowledge can be systematically transformed from unstructured clinical guidelines into a machine-interpretable representation that supports consistent and traceable decision-making. Through scenario-based evaluation, the approach shows its ability to produce safe and explainable outputs across a range of rehabilitation conditions, highlighting the effectiveness of combining symbolic reasoning with controlled use of language models.

While the current implementation is limited in scope, the modular design provides a strong foundation for future extension in terms of protocol coverage, personalisation, and scalability. More broadly, this work demonstrates the viability of neuro-symbolic approaches in safety-critical domains, where reliability and interpretability are essential. By emphasising structured knowledge, deterministic logic, and grounded explanations, the system offers a practical and extensible framework for developing trustworthy AI-driven decision-support tools.

11 AI Tool Declaration

To enhance clarity and accelerate development, AI tools were used as supportive aids during the project. GPT-5.3 was used for grammatical refinement, code review, and code optimization. Gemini 3 Flash – Fast was used to support scenario generation. Claude Sonnet 4.6 was used to support coding assistance and scenario generation.

All foundational concepts, including the system architecture, data modelling, rule design, and final implementation strategy, were developed and validated by the project team. The final design decisions and project responsibility remain solely with the team.

12 Reference

Insights10 (n.d.). Singapore Physiotherapy Market Report 2022 to 2030. Page 5. https://www.slideshare.net/slideshow/singapore-physiotherapy-market-analysis-sample-report/255596937

Abdul Wasay (April 2026) Artificial Intelligence (AI) Enhanced Remote Physiotherapy Market Report 2026. Page 5 – 7. https://www.thebusinessresearchcompany.com/report/artificial-intelligence-ai-enhanced-remote-physiotherapy-global-market-report

Appendix A – Project Proposal

Graduate Certificate in Intelligent Reasoning Systems
Practice Module Proposal

The WellnessBot

By
The Stacks (Group 7)

Loo Kin Xing | Sherwin Tang Wei Hao | Foo Chit Yan | Foo Swee Yoong
A0073762B | A0125052W | A0341007W | A0341027R

Institute of System Science
National University of Singapore

1 Introduction

1.1 Overview

This project develops a system to support recovery after post-arthroscopic knee surgery, providing safe and easy-to-understand exercise and self-care recommendations through a chat interface.

The system collects key information such as recovery stage, pain level, and symptoms, and uses it to determine the user's condition and provide appropriate exercise and self-care recommendations. Each recommendation is supported with clear explanation and evidence, helping user understands the reasonings behind it while ensuring transparency and reliability. The system also includes a feedback and improvement loop, where user interactions are analysed to identify issues and continuously improve future recommendations.

The scope focuses on post-arthroscopic knee surgery as a representative rehabilitation scenario, allowing the system design to be developed and evaluated effectively within project timeframe.

1.2 Importance & Relevance

Post-arthroscopic knee rehabilitation requires safe, stage-appropriate guidance, yet patients often lack clear and reliable support outside clinical settings. AI is a powerful tool for rehabilitation support, but for patients to feel confident using it at home, the reasoning behind each recommendation must be clear and easy to understand.

The novelty of this project lies in its transparent, decision-first approach, where recommendations are made using explicit rules and structured knowledge instead of opaque models, with the language model used only for interpretation and explanation. It also introduces an offline knowledge loop, where interaction logs and user feedback are analysed to refine rules and improve the system in a controlled and traceable manner.

1.3 Goal & Objectives

The goal of this project is to develop a system that provides safe, personalized, and easy-to-understand exercise and self-care recommendations to support recovery for post-arthroscopic knee surgery patients.

To achieve this, the system is designed with following objectives:

- To be able to understand user inputs through a chat-based interface, assess the user's recovery condition, and generate reliable and appropriate recommendations.
- To deliver clear and easily understandable explanations to user
- To be able to analyse user's feedback and continuously improve its recommendations over time.

2 Market Research

2.1 Background

The growing demand for physiotherapy services in Singapore is not adequately met due to a shortage of physiotherapists, creating gaps in timely access to physio care (Insights10, n.d.).

In addition, the high cost of physiotherapy service, ranging from $50 to $400 per session, can pose a financial burden for many patients (Insights10, n.d.).

For individuals with mobility issues, accessibility is also a challenge for them as they may find it difficult to travel from home to treatment centres.

On top of that, patients often face difficulties in obtaining timely clinical advice, as support hotlines are frequently engaged, limiting their immediate access to professional guidance.

2.2 User Needs & Market Trends

There is a growing demand for digital healthcare solutions that address key user needs, including access to remote guidance without the need for in-person visits, real-time advice that is available anytime and anywhere, and personalized therapy that adapts to individual recovery progress (Abdul Wasay, 2026).

At the same time, emerging market trends are shaping the development of such systems.

Regulatory frameworks like the EU AI Act (2024) are driving the need for explainable and transparent clinical AI, while Asian Bioethics Review (2025) also highlights that explainability is one of the most important factors influencing user acceptance of AI in healthcare.

In addition, local regulations such as the Personal Data Protection Act in Singapore introduce stricter guidelines for data protection and AI system governance, further emphasizing the importance of building trustworthy, compliant, and user-centric wellness solutions.

2.3 Market Overview and Opportunities

According to Artificial Intelligence (AI) Enhanced Remote Physiotherapy Market Report 2026 (Abdul Wasay, 2026), AI Enhanced Remote Physiotherapy market size has reached to $2.12 billion in 2025 and it is expected to grow to $6.44 billion in 2030 at a compound annual growth rate (CAGR) of 24.8%, with North America was the largest region in 2025 and Asia-Pacific is the fastest growing region in forecast (Abdul Wasay, 2026).

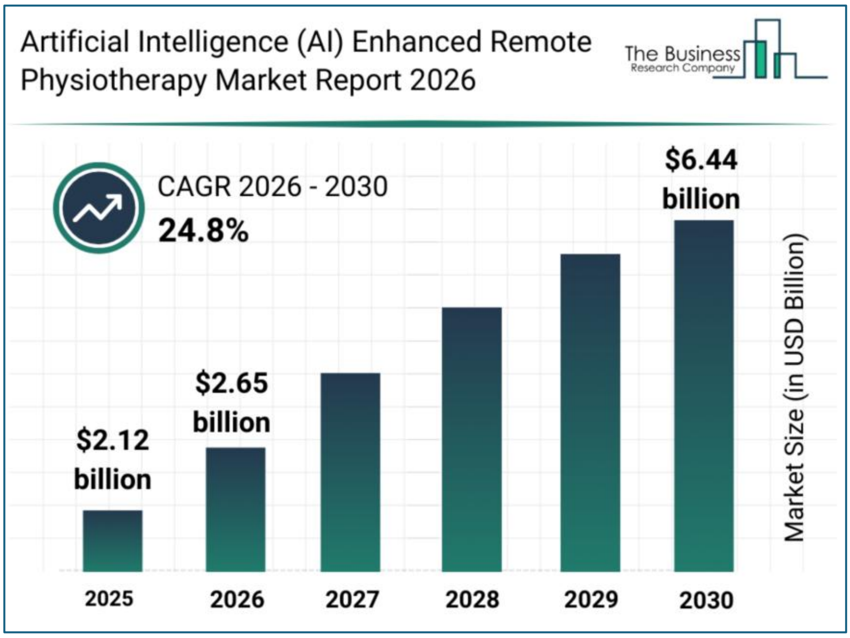


Figure 1: Global Artificial Intelligence (AI) Enhanced Remote Physiotherapy Market Size Gain ($ Billion), 2025 – 2030

The AI enhance remote physiotherapy market can be divided by five segments (Abdul Wasay, 2026) and table below shows how WellnessBot is positioned in each segment.

Market Segment	**WellnessBot**
By Component	Software
By Therapy	Musculoskeletal
By Deployment	In-App (non-cloud)
By Application	Rehabilitation
By End-User	Homecare settings

Table 1: WellnessBot in Market Segment

From the market segments listed above, it is clearly seen that WellnessBot has the potential opportunities to incorporates other types of musculoskeletal injuries such as ligament tears or rheumatoid arthritis. It can also be further developed for other application such as ergonomic exercise and other user settings such as fitness centre.

2.4 Key Players Comparison

There are plenty of key players in AI enhanced remote physiotherapy market, and table below shows some of the comparisons of WellnessBot versus these key players.

Key Player	**How it is compared to WellnessBot**
Hinge Health which focuses on rehabilitation AI	No post-surgical constraint filtering
Physitrack which provides exercise libraries	No conversational AI or safety filtering
Sword Health which provides AI physical therapy	Use of wearable sensors unlike device-free WB
Kaia Health which focuses on digital treatment for musculoskeletal conditions	Use of camera which might lower user's acceptance

Table 2: Comparing WellnessBot with other Key players

3 Project Scope

3.1 Academic

This system demonstrates all four AI systems paradigms from course criteria in a single integrated system:

- Decision Automation → Rule & Inference Engine (RECOMMEND / FORBID / ESCALATE)
- Business Resource Optimisation → Intervention Planner (weighted multi-criteria scoring as informed search)
- Knowledge Discovery → LLM semantic extraction, RAG retrieval, structured interaction logs
- Cognitive / Knowledge Systems → Knowledge Graph, User State Inference, LLM response generation, full chatbot

It contributes a two-layer safety design pattern (hard constraint pre-filter + soft scoring), which is also applicable to other safety-critical recommendation domains.

The audit log bridges rule-based explainability which is academically relevant to clinical AI governance debate.

3.2 Clinical

From clinical domain perspective, this application targets:

- single surgery which is arthroscopic knee surgery
- adult user who is 0 to 20 weeks post-operation which is a standard recovery trajectory
- six weeks of rehabilitation phases - Initial, Early, Intermediate, Late, Transitional and Return-to-Sport
- single best exercise recommendation per session

3.3 Limitations

The system has several limitations that should be acknowledged. The knowledge graph is pre-populated and does not support real-time updates, requiring manual intervention whenever clinical guidelines change.

In addition, the system does not incorporate online learning; instead, user interaction logs are stored for future offline analysis and improvement.

Furthermore, the system relies on subjective and unverified user inputs, which could affect the reliability of recommendations.

Recovery progression is also determined using a time-based phase mapping approach rather than criterion-based assessment.

4 System Design

4.1 Overall Architecture

The system adopts a neuro-symbolic hybrid approach that combines the strengths of both symbolic and neural methods.

The symbolic layer, which includes components such as a knowledge graph, rule engine, and planner with hard constraints, ensures safety, determinism, and full explainability of the system's decisions.

In contrast, the neural layer—comprising large language models (LLMs) and retrieval-augmented generation (RAG) embeddings—enables natural language understanding, flexible response generation, and effective semantic retrieval.

Importantly, the symbolic components act as a governing layer that gates and constrains the actions of the neural components, ensuring that the system does not generate clinically unsafe outputs while still maintaining adaptability and user-friendly interactions.

4.2 System Flow Chart

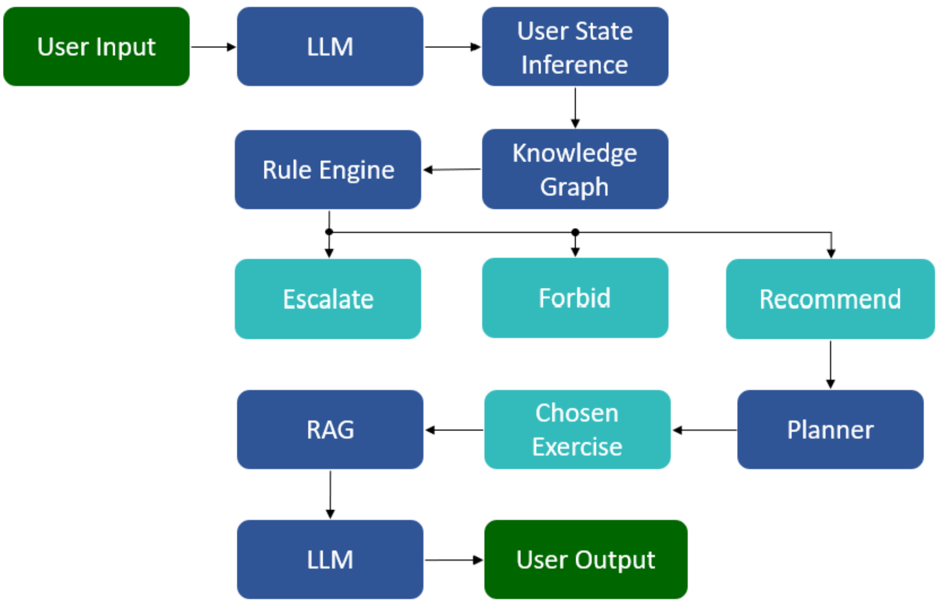


Figure 2: System Flow Chart (Real-Time)

The system design uses sequential pipeline below:

User input → LLM Extraction → User State Inference → Knowledge Graph → Rule Engine → Intervention Planner → RAG → LLM Response → User Output

The Rule & Inference Engine is the primary safety gate, producing one of three deterministic outcomes: RECOMMEND / FORBID / ESCALATE, but only RECOMMEND passes candidates to the Planner.

Planner is framed as informed search based on scoring of exercise priority, equipment availability & exercise history.

Retrieval for Augmented Generation will be a 1 to 1 retrieval from the database, with possibility to expand using BM25.

The symbolic layer always executes before the neural layer, with neural components operate within a pre-validated decision space and cannot produce an unsafe output unchecked.

Every decision is fully auditable, where scores, eliminations, and selected exercise are emitted to the structured interaction log, for future refinement.

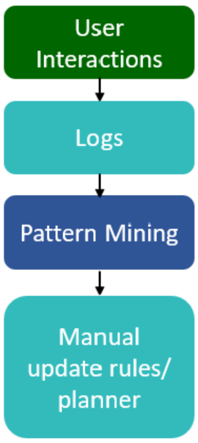


Figure 3: System Flow Chart (Offline)

The system logs interaction data for the purpose of pattern mining and continuous improvement. This analysis is conducted at multiple levels, including rule-coverage analysis, decision-level analysis, and exercise-level analysis, to evaluate how effectively the system performs.

Through this process, several key patterns are identified, such as rule misfires where system decisions do not match expected outcomes, and frequent "ESCALATE" responses that may indicate overly strict rules.

In addition, unhandled exercises are analyzed to uncover gaps in the knowledge graph, while user dissatisfaction signals, such as negative feedback (e.g., thumbs down), are used to highlight areas requiring refinement. Together, these insights support systematic enhancement of the system's accuracy, coverage, and user experience.

5 Database

5.1 Data Collection

The database for WellnessBot is built from published patient guidelines from clinical sources such as hospital, clinics, medical centres and clinical professionals.

This system focuses only on post-arthroscopic knee surgery rehabilitation protocol, where the overall protocol is built based on the published guideline from Massachusetts General Hospital, and the step-by-step instructions are built from other clinical sources.

Every single data populated into database is cited with source link as evidence.

5.2 Data Preparation

The database is built first on Excel format for easier population based on domain knowledge. It is then simplified & restructured where rehabilitation phases, exercises and evidences are labelled with IDs for rule mapping. Both databases are maintained for debug purpose and future expandability. The databases are then converted into JSON format for downstream implementation.

5.3 Dataset Overview

Below is the overview of current dataset structure.

- Number of Protocol: 1
- Phases per Protocol: 6 (where final phase is defined as Return-to-Sport and no step-by-step instruction will be given)
- Exercise candidates: 25
- Instruction chunks per exercise: 1 to 2
- Exercise priority rating: 1 to 3 (where 1 must be prioritized first)
- Equipment options: Stool, Chair, Towel, Resistance band, Bicycle etc
- Red flag rule: 9
- Self care rule: 10 (both red flag and self-care rule will be integrated eventually)
- Test Scenarios: 10 with 3 Escalate, 2 Forbid, 5 Recommend (will be expanded to 20 to 30 scenarios and simulate using 2 to 3 LLM model)

5.4 Dataset Challenges

Several challenges were identified in developing the system, along with corresponding strategies to address them.

First, the definition of recovery phases varies across different clinical guidelines, which may lead to inconsistencies; to mitigate this, a single standardized protocol from a renowned hospital is adopted.

Second, physical constraints can differ even within the same recovery phase, so the early phase is further divided into sub-phases, with each exercise labelled and prioritized accordingly.

Third, general self-care protocols are often scattered across multiple sources, making them difficult to interpret; therefore, information is consolidated from various webpages to derive structured self-care rules based on recovery phases.

Lastly, since many exercises have progressive variations that may not always be safe for all users, progressive suggestions are avoided as a precaution to ensure patient safety.

6 Implementation

6.1 Design Practicality

In real-world system design, there is a strong emphasis on maintainability, deployment feasibility, and avoiding overly complex infrastructure.

To align with these principles, the system adopts a modular implementation approach, where each component can be developed and tested independently, improving maintainability and ease of updates.

For deployment feasibility, an incremental integration approach is used, allowing components to be gradually combined and validated, thereby reducing risk during system rollout.

Additionally, to avoid complex infrastructure, the system follows a sequential pipeline approach, ensuring a simpler, more manageable architecture while still maintaining effective functionality.

6.2 Challenges

Several technical challenges were identified in the system design, along with corresponding strategies to address them.

To ensure API key security, environment variables are used to support secure deployment practices.

For data structuring in retrieval-augmented generation (RAG), content is converted into chunk-based JSON format to enable consistent and reliable retrieval.

To reduce retrieval design complexity, a 1-to-1 key-based retrieval approach is adopted, simplifying the system architecture while allowing for future expansion to more advanced methods such as BM25 retrieval.

Additionally, for data structuring in LLM-based natural language understanding (NLU) extraction, carefully designed input prompts are implemented to ensure that extracted information is reliable and can be effectively utilized by downstream components such as the rule engine and planner.

7 Results and Progress

7.1 Result Screenshot

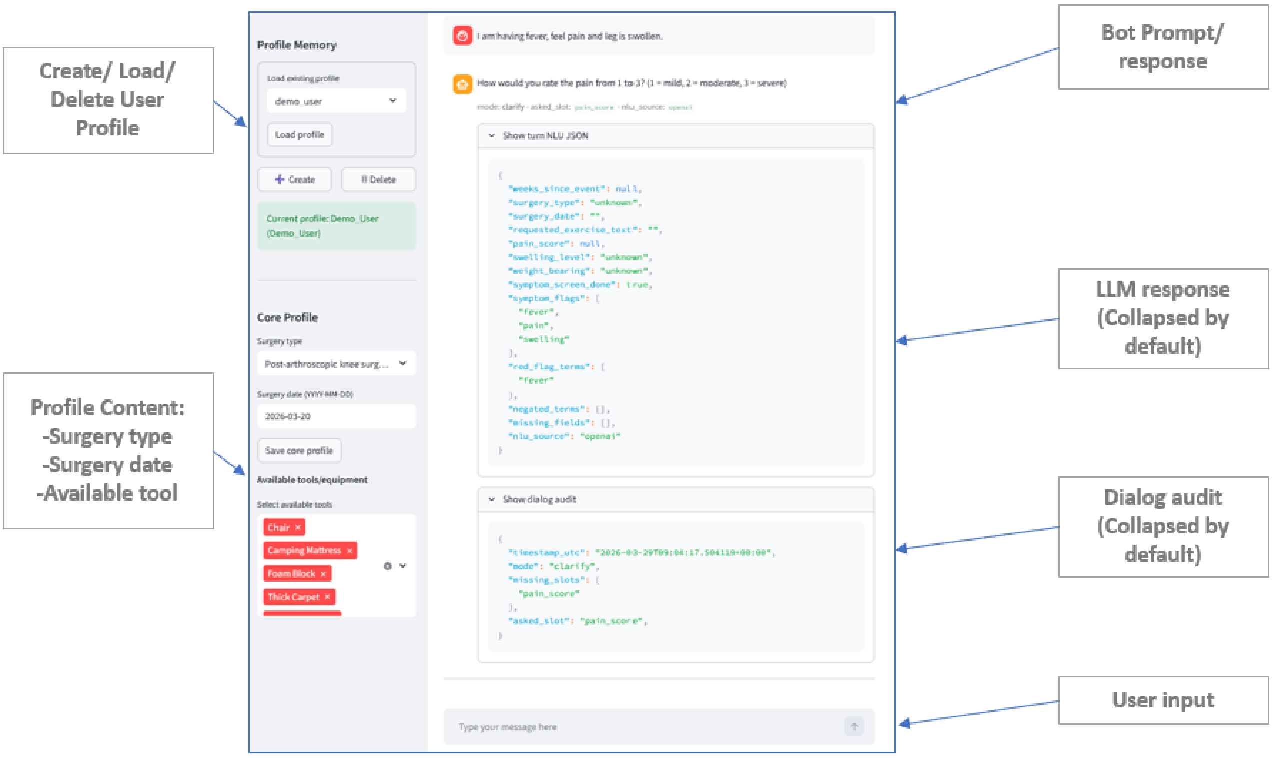


Figure 4: Screenshot of preliminary user interface

7.2 Academic & Market Relevance

The system is currently able to convert user input into structured JSON format and produce 3 deterministic decisions: Recommend, Forbid and Escalate, plus retrieve rehabilitation content for explanation.

The results show academic relevance via deterministic decisions based on explicit rules, with rehabilitation knowledge organized through a knowledge graph.

To enhance factual consistency, Retrieval-Augmented Generation (RAG) is integrated alongside an LLM-based NLU.

This system also shows market relevance by ensuring decisions are rule-based, traceable, and auditable, aligning directly with the MOH healthcare AI context.

7.3 Deviation & Impacts

To optimize the system, several key deviations were implemented within the RAG framework.

First, the retrieval method shifted from vector-based to key-based retrieval because the former requires significant infrastructure setup; this change preserves the core pipeline while significantly improving deployment feasibility.

Second, RAG's role was refocused from active decision-making to explanation retrieval. This pivot ensures safer decision-making by avoiding LLM hallucination risks, effectively demonstrating a robust neuro-symbolic AI architecture that fosters user trust.

Finally, the system moved from web content retrieval to JSON format retrieval, utilizing chunking to improve retrieval accuracy and system efficiency. Collectively, these adjustments ensure tighter alignment between retrieved data and the explanations provided to the user.

7.4 Performance Evaluation

The system is evaluated using two layers of performance evaluation.

7.4.1 Layer 1 - Decision-Level Confusion Matrix

The first layer is to assess whether the system selects the correct action (Recommend/ Forbid/ Escalate). There will be four evaluation metrics:

- **Accuracy** – Overall proportion of correct decisions across all scenarios
- **Precision** – Measures how reliable each predicted decision is (e.g., when the system predicts ESCALATE, how often it is correct)
- **Recall (Critical for safety)** – Measures how well the system detects all true cases of a decision
- **F1-Score** – Balances precision and recall for overall performance

This layer serves as the primary evaluation, as it directly reflects the correctness and safety of the system's decision-making.

Table 3 presents representative scenarios to identify correct predictions and failure cases, such as missed escalation or overly strict forbidding.

Table 4 summarizes these outcomes in a confusion matrix, providing an overall distribution of predicted versus actual decisions.

Scenario	Expected	System	Result
Phase 2, user having fever	ESCALATE	ESCALATE	Pass
Phase 3, Pain score is 1, swell score is 0	RECOMMEND	RECOMMEND	Pass
Phase 2, wound is bleeding	ESCALATE	RECOMMEND	Fail
Phase 4, Pain score is 1, swell score is 1	RECOMMEND	FORBID	Fail
Phase 1_2, Pain score is 0, swell score is 2	RECOMMEND	RECOMMEND	Pass

Table 3: Pass/Fail Sample Scenarios for Recommend/ Forbid/ Escalate

	Predict: RECOMMEND	Predict: FORBID	Predict: ESCALATE
Actual: RECOMMEND	2	1	0
Actual: FORBID	0	0	0
Actual: ESCALATE	1	0	1

Table 4: Decision-Level Confusion Matrix (Predicted vs Actual)

7.4.2 Layer 2 - Exercise-level Evaluation

This layer is to evaluate whether the recommended exercise or self-care routine is appropriate for the inferred rehab phase and user condition. It serves as a secondary validation following correct decision selection.

Scenario	Expected	System	Result
Phase 3, Pain score is 1, swell score is 0	Quadriceps Stretch, Standing	Quadriceps Stretch, Standing	Pass
Phase 1_2, Pain score is 0, swell score is 2	Knee flexion and extension	Wall slides	Fail

Table 5: Pass/Fail Sample Scenarios for Recommended Exercise

The planner selects a single exercise using an informed search strategy guided by priority rules, available tools, and exercise history. While multiple exercises may be valid, only one is selected.

Due to this single-recommendation setup, the evaluation primarily reflects overall correctness.

Both exercise and self-care recommendations are evaluated consistently under the same framework, based on the user's rehabilitation phase and condition.

7.5 Other Challenges

Below are the other challenges and the strategies to overcome them.

7.5.1 Handling incomplete and uncertain user input

User inputs may be missing, unclear, or inconsistent, affecting decision accuracy.

Strategy: Use a chat loop to collect key information progressively and apply validation for critical fields.

7.5.2 Knowledge coverage and generalisation

The system is limited to a specific rehabilitation scenario, which may not cover all cases.

Strategy: Design the knowledge structure to be modular and extendable for future conditions.

7.5.3 Balancing safety and user expectations

Strict rule-based decisions may limit flexibility and reduce user satisfaction.

Strategy: Provide safe alternatives with clear explanations to maintain both safety and user trust.

7.5.4 Limited time for knowledge refinement

Full knowledge loop implementation and data collection are time constrained.

Strategy: Implement a lightweight analysis using logs and feedback, and position full refinement as future work.

7.5.5 Integration complexity

Integrating multiple components increases system complexity and debugging difficulty.

Strategy: Maintain a modular design and test components independently before integration.

7.6 Overall Progress

Milestone 1 — Foundation & Schema Alignment (100%)

- Finalise data schema for all input fields with valid ranges
- Complete KG node and relationship design on paper before implementation
- Define all API contracts and interface specifications between modules
- Gather and annotate evidence chunks linked to exercises and phases
- Finalize NLU output JSON format that satisfies KG query requirements

Milestone 2 — Core Symbolic Engine (85%)

- Build and populate KG with exercises across 5 phases
- Implement rule engine with safety rules and scoring
- Build input pipeline with validation, sanitisation, and KG mapping
- Implement rule score logging schema
- Validate KG and rule engine against clinical scenarios provided by domain expert

Milestone 3 — Planner & LLM Modules (10%)

- Implement informed search planner
- Validate planner exercise selections against domain scenarios
- Build NLU extraction prompts with few-shot examples and output validation
- Build RAG retrieval pipeline over evidence chunks
- Engineer RAG explanation prompts with citation

Milestone 4 — Integration & UI (20%)

- Build conversational chat UI with input forms and recommendation display
- Integrate input pipeline, KG, rule engine, and planner end-to-end
- Connect NLU output to KG query layer
- Connect planner output to RAG explanation module
- Display evidence citations, dosage explanations, and red flag warnings in UI

Milestone 5 — Evaluation & Knowledge Loop (0%)

- Run full system against scenarios and edge case test set
- Measure rule engine precision, planner effectiveness, NLU accuracy, RAG citation quality
- Validate system performance with confusion metrics
- Run offline pattern mining on logged rule scores and refine KG if needed
- Final end-to-end integration testing and bug resolution

Milestone 6 - Final Demo & Presentations (0%)

- Prepare representative demo scenarios
- Ensure UI clearly communicates system reasoning and evidence citations
- Document architecture, API contracts, and module interfaces
- Final team review and presentation readinessEvery single data populated into database is cited with source link as evidence.

8 Conclusion

This project presents a decision-support system for post-arthroscopic knee surgery rehabilitation, combining structured knowledge, rule-based reasoning, and a planner to deliver safe and explainable recommendations.

It demonstrates a neuro-symbolic approach, where decisions are made using transparent logic while a language model is used only for interpretation and explanation.

The project contributes academically by showing how explainable, deterministic AI systems can be designed for safety-critical applications.

Overall, the project bridges the gap between academic research and real-world application by translating structured rehabilitation knowledge into a usable, trustworthy system.

9 AI Tool Declaration

To enhance clarity and accelerate development, GPT-5.3 was used as a supportive tool for grammatical refinement and code optimization. All foundational concepts, including system architecture and data modelling, are original work. The final implementation strategy and design remain the sole responsibility of the project team.

10 References

Insights10 (n.d.). Singapore Physiotherapy Market Report 2022 to 2030. Page 5. https://www.slideshare.net/slideshow/singapore-physiotherapy-market-analysis-sample-report/255596937

Abdul Wasay (April 2026) Artificial Intelligence (AI) Enhanced Remote Physiotherapy Market Report 2026. Page 5 – 7. https://www.thebusinessresearchcompany.com/report/artificial-intelligence-ai-enhanced-remote-physiotherapy-global-market-report

Appendix B – Mapped System Functionalities

Paradigm	System Functionality in WellnessBot	Knowledge, Techniques & Skills Applied	Course
Decision Automation	Rule Engine (ESCALATE / FORBID / RECOMMEND)	Deterministic rule-based inference, constraint enforcement	MR
Business Resource Optimisation	Planner (exercise selection)	Constraint-based selection and structured decision control	RS
Knowledge Discovery / Processing	RAG retrieval + NLU extraction + Offline Knowledge Loop	Evidence retrieval, structured extraction, and log-guided pattern analysis for manual refinement	CGS + RS
Cognitive / Knowledge Systems	Knowledge Graph + State Inference	Knowledge representation, state modelling	MR + CGS

Appendix C – Installation and User Guide

WellnessBot

Installation & User Guide

Version: v2.0

Application: WellnessBot — Rehabilitation Decision Support

Document Type: Appendix — Installation & User Guide

1. System Overview

WellnessBot is a neuro-symbolic hybrid AI chatbot that provides patients with contextually appropriate, evidence-grounded exercise recommendations during arthroscopic knee rehabilitation. It adapts dynamically to each user's surgery type, recovery phase, reported symptoms, available equipment, and exercise history.

Pipeline Architecture

```
User Input
→ [1] NLU Extraction        (OpenAI GPT-4o-mini)
→ [2] Dialog Merge          (Slot filling into ConversationState)
→ [3] CLARIFY Check         (Missing slots trigger question loop)
→ [4] User State Inference  (Derives phase_id, risk_flags)
→ [5] Rule Engine           (RECOMMEND / CLARIFY / FORBID / ESCALATE)
→ [6] Intervention Planner  (Exercise selection — RECOMMEND path only)
→ [7] RAG Retrieval         (BM25 evidence chunk lookup)
→ [8] LLM Response Generation (Patient-facing validated output)
→ User Output (Streamlit chat UI)
```

2. Prerequisites

Before installing WellnessBot, ensure the following are available on your machine:

Requirement	Minimum Version	Notes
Python	3.10+	Check with python --version
pip	Latest	Bundled with Python
OpenAI API Key	—	OpenAI API Key is required for full system functionality
Internet connection	—	Internet connection is required for OpenAI API calls

Note: WellnessBot has been tested on Windows 11. It is compatible with macOS and Linux, with minor command differences noted where applicable.

3. Installation

3.1 Extract the Zip Archive

The submission is delivered as a zip file. Extract it to a location of your choice.

After extraction, the folder layout will look like this:

IRS-PM-2026-01-24-GRP7-WellnessBot/

```
├── ProjectReport/
│   └── Installation_and_User_Guide  ← this document
└── SystemCode/  ← all runnable code lives here
    ├── .env.example
    ├── requirements.txt
    ├── run_streamlit.ps1
    ├── run_tests.ps1
    ├── data/
    ├── logs/
    └── wellnessbot/
```

Important: All terminal commands in this guide must be run from inside the SystemCode/ folder unless otherwise stated.

Open a terminal (PowerShell on Windows, or Terminal on macOS/Linux) and navigate there:

Windows (PowerShell):

```
cd path\to\IRS-PM-2026-01-24-GRP7-WellnessBot\SystemCode
```

macOS / Linux:

```
cd path/to/IRS-PM-2026-01-24-GRP7-WellnessBot/SystemCode
```

3.2 Create a Virtual Environment

It is strongly recommended to use a virtual environment to isolate dependencies. Run the following from inside SystemCode/:

Windows:

```
python -m venv .venv

.venv\Scripts\activate
```

macOS / Linux:

```
python -m venv .venv

source .venv/bin/activate
```

You should see (.venv) prepended to your terminal prompt once the environment is active.

3.3 Install Dependencies

With the virtual environment active, install all required packages:

```
pip install -r requirements.txt
```

This installs the following packages:

Package	Version	Purpose
streamlit	≥ 1.36.0	Web UI framework
openai	≥ 1.40.0	LLM API client (GPT-4o-mini)
pydantic	≥ 2.7.0	Data validation and schema enforcement
python-dotenv	≥ 1.0.1	Environment variable loading from .env
pyyaml	≥ 6.0.1	YAML configuration parsing
pytest	≥ 8.2.0	Test suite runner
ruff	≥ 0.6.0	Code linter

3.4 Configure Environment Variables

The .env file must be created inside SystemCode/ (alongside requirements.txt). Create it by copying the provided example:

Windows (PowerShell):

```
Copy-Item .env.example .env
```

macOS / Linux:

```
cp .env.example .env
```

Open SystemCode/.env in a text editor and set the required values:

```
APP_VERSION=v2.0

LOGGING_ENABLED=1

LOG_DIR=./logs

OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

OPENAI_MODEL=gpt-4o-mini
```

Security: Please do not share your OPENAI Key

4. Running the Application

Note: All commands below assume your terminal is inside SystemCode/ with the virtual environment active.

4.1 Windows (PowerShell)

A convenience script is provided that automatically sets PYTHONPATH and launches Streamlit. Run it from inside SystemCode/:

```
.\run_streamlit.ps1
```

4.2 macOS / Linux

From inside SystemCode/, with the virtual environment active:

```
PYTHONPATH=.. streamlit run wellnessbot/Wellnessbot.py
```

Accessing the Application

Once started, open your web browser and navigate to:

```
http://localhost:8501
```

Streamlit will also print the URL to the terminal. The application loads immediately — no build step is required.

5. User Guide

5.1 Launching the App

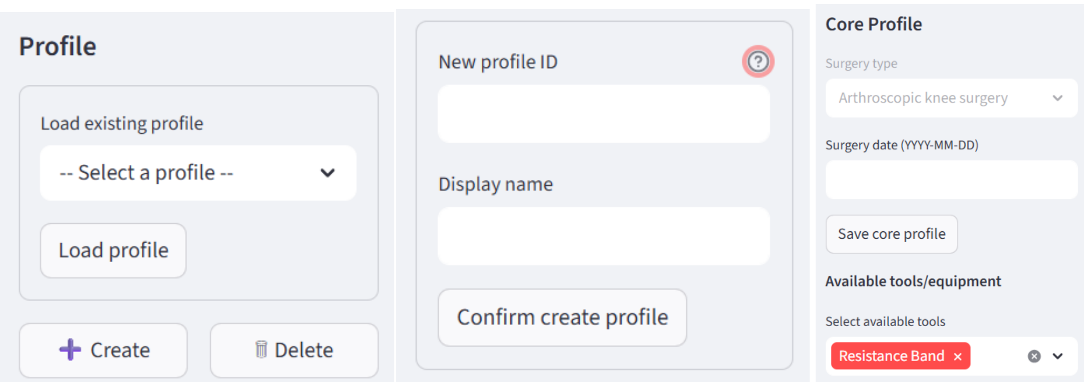
After running the start command (see section 4), open http://localhost:8501 in your browser. You will see the WellnessBot chat interface with a sidebar on the left.

5.2 Creating a User Profile

Before starting a conversation, you must select or create a user profile. The profile stores your surgery details and equipment list, so they persist across sessions.

Steps:

1. In the left sidebar, locate the Profile section.
2. If no profiles exist, click Create.
3. Enter the Display Name, then click "Confirm create profile" to initialise the profile.
4. After the profile is created, input the Surgery Date and Equipment Available in the profile settings.
5. To switch profiles, select the desired profile from the dropdown menu in the sidebar.



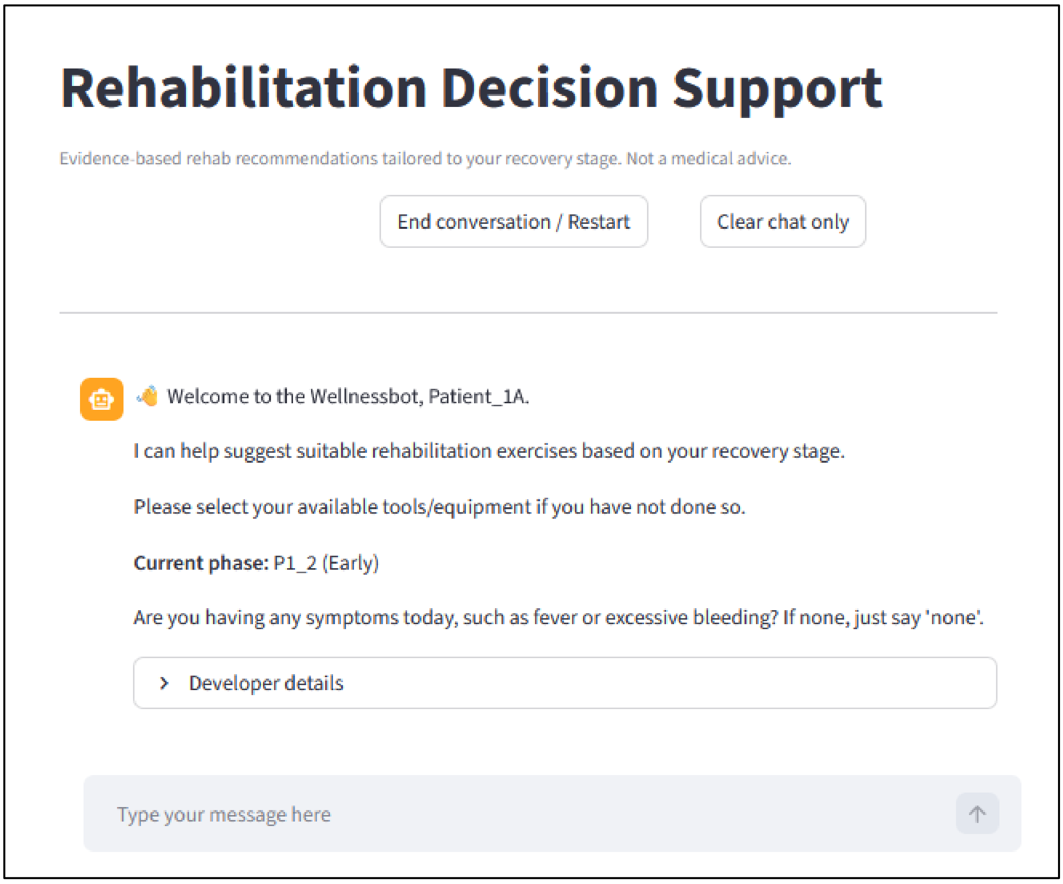
Profile data is stored locally in SystemCode/data/profiles/<profile_id>.json. It is not transmitted anywhere other than the OpenAI API for NLU processing.

5.3 Starting a Conversation

With a profile selected, type your message in the chat input box at the bottom of the screen and press Enter or click Send.

Example opening messages:

- "None"
- "I am having slight fever and no bleeding"



The bot will respond based on the information it has collected so far. If information is missing, it will ask follow-up questions.

5.4 Slot-Filling Dialogue Flow

WellnessBot collects the required user information before generating any exercise recommendation. If any required field is missing, the system issues a targeted follow-up question to ensure completeness of the input state.

For example, if the surgery date has not been provided during profile setup, WellnessBot will prompt the user to enter the surgery date before proceeding to the symptom assessment stage.

Slot	What the Bot Asks	Example Answer
Surgery date / weeks	"When was your surgery or injury? Please tell me the date (YYYY-MM-DD) "	"2025-12-01"
Symptoms	"Are you having any symptoms today, such as fever or excessive bleeding? If none, just say “none”."	"I am having fever" / "None"
Pain score	"How would you rate the pain from 0 to 3?"	Numeric input (e.g., 1) or selection via UI buttons
Swelling score	"How would you rate the swelling from 0 to 3? "	Numeric input (e.g., 0) or selection via UI buttons

Tips:

- If you have already set your surgery type and date in your profile, the bot may skip those questions.
- The bot remembers your answers within a session and will not re-ask completed slots.

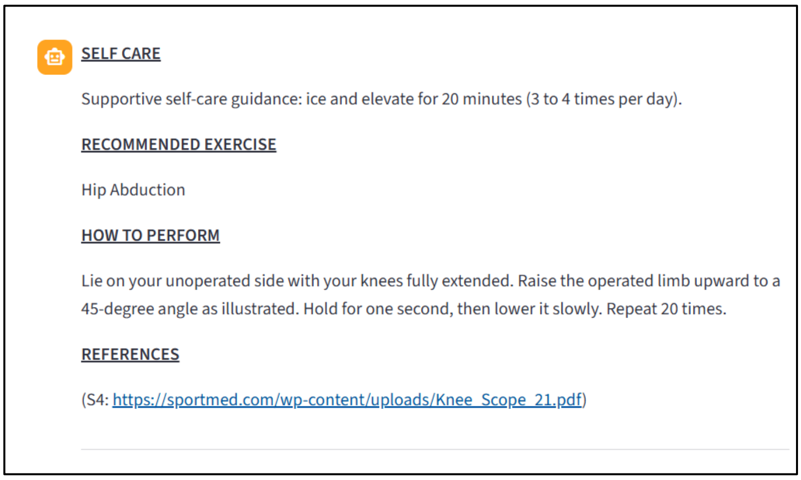
5.5 System Responses

Once sufficient information is collected, WellnessBot generates one of four responses:

RECOMMEND — Exercise Recommendation

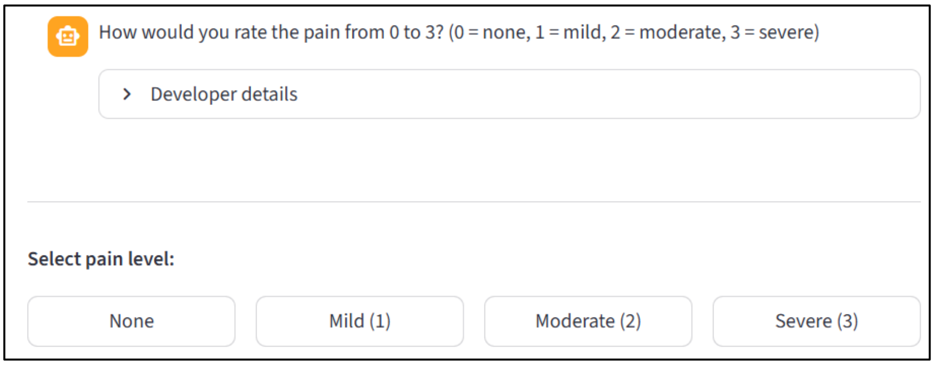
The most common response. The bot provides:

- Self Care (if applicable) — ice/elevation advice based on your symptoms
- Recommended Exercise — exercise name, position, equipment needed, and caution notes
- How To Perform — one grounded instruction line from the clinical evidence database
- References — source citations with links



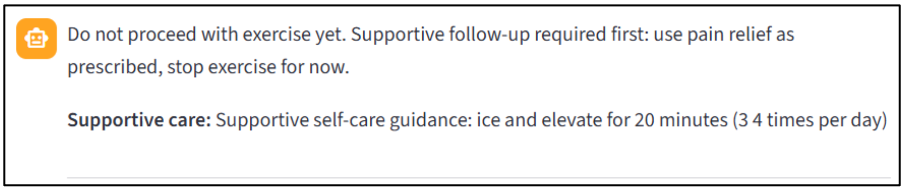
CLARIFY — Follow-up Question (Internal Step)

The bot asks for one missing piece of information before it can make a recommendation. A list of possible exercises may also be shown.



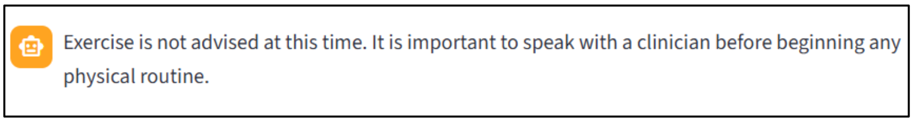
FORBID — Exercise Blocked

The bot explains why an exercise cannot be recommended at this time (e.g., your swelling score is too high). No exercise is provided.



ESCALATE — Seek Clinical Help

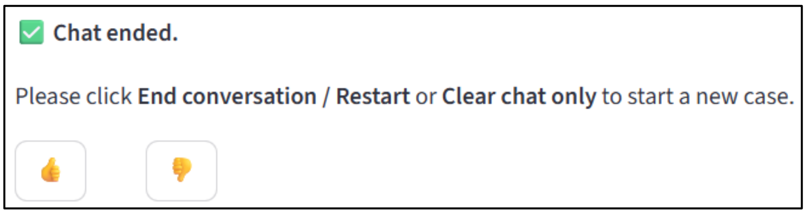
If red-flag symptoms are detected (e.g., fever, active bleeding, extreme swelling), the bot advises you to contact your healthcare provider immediately. No exercise is recommended.



5.6 Providing Feedback

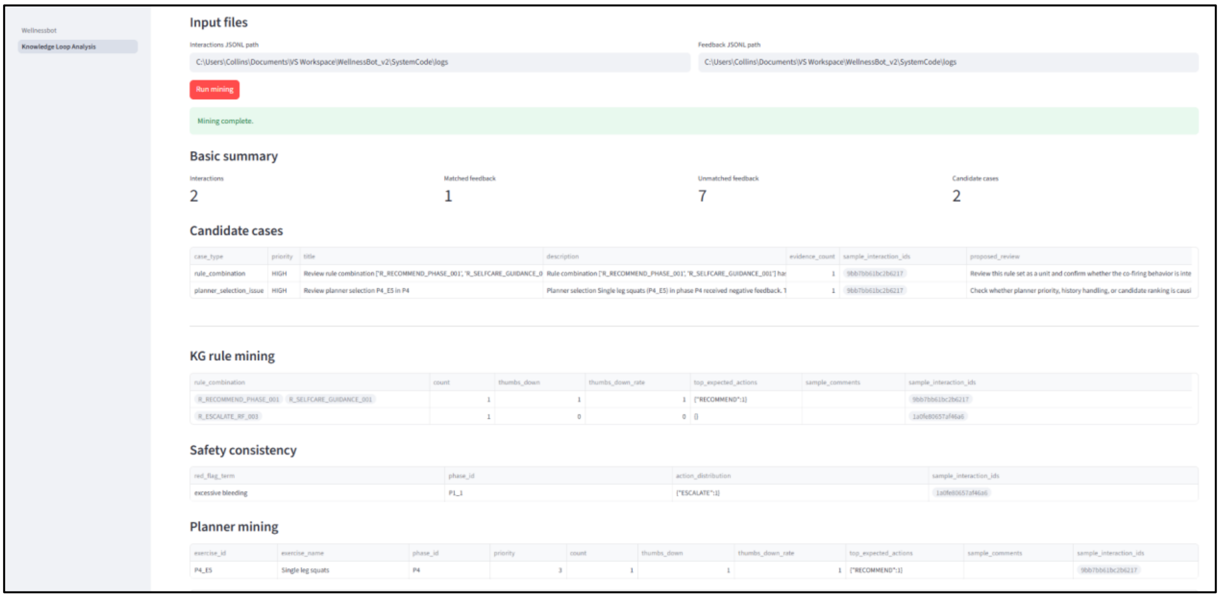
After each final system response (RECOMMEND, FORBID, or ESCALATE), a feedback prompt is presented to the user in the form of a thumbs-up or thumbs-down selection. Users may indicate whether the response was helpful or not through this interface.

All feedback is recorded as part of the system's audit trail, together with the corresponding decision, input state, and reasoning outputs. This information is subsequently used for developer analysis to assess system performance, identify failure patterns, and support targeted refinement of the knowledge graph, rule set, and planner logic through the offline knowledge loop.



5.7 Knowledge Loop Analysis

The Knowledge Loop Analysis page is used to evaluate system behaviour using interaction logs and user feedback.



Steps:

1. Select Input Files

   Ensure the interaction logs and feedback logs are correctly loaded.
2. Run Mining

   Click "Run Mining" to process logs and generate results.
3. Review Basic Summary

   View key metrics: total interactions, feedback coverage, and refinement candidates.
4. Analyse Refinement Candidates

   Review flagged cases with issue summaries, interaction IDs, and suggested actions.

   These highlight potential problems such as rule conflicts, negative feedback patterns, or suboptimal planner decisions.
5. Deep Dive into Mining Views
   - KG Rule Mining: Identify problematic rule combinations
   - Safety Consistency: Check if red-flag conditions trigger correct actions
   - Planner Mining: Evaluate exercise selection behaviour

This workflow supports systematic identification of system weaknesses and guides manual, developer-driven refinement of the knowledge graph, rules, and planner, without affecting runtime logic.

6. Environment Variables Reference

Variable	Required	Default	Description
OPENAI_API_KEY	Yes	—	OpenAI API key for NLU and LLM response generation
OPENAI_MODEL	No	gpt-4o-mini	OpenAI model for NLU extraction
LOGGING_ENABLED	No	1	Set to 0 to disable interaction logging
LOG_DIR	No	./logs	Directory for daily JSONL interaction log files
APP_VERSION	No	v2	Version tag written into audit log entries

7. Troubleshooting

7.1 Script not found / CommandNotFoundException

Ensure you are running the command from inside SystemCode/, not from the repo root or elsewhere:

```
cd path\to\IRS-PM-2026-01-24-GRP7-WellnessBot\SystemCode .\run_streamlit.ps1
```

7.2 PowerShell script execution blocked

Run this once to allow local scripts, then retry:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

7.3 Application does not start

- Ensure your virtual environment is active ((.venv) in your terminal prompt).
- Ensure all dependencies are installed: pip install -r requirements.txt.
- Ensure you are running the command from inside SystemCode/.

7.4 ModuleNotFoundError: No module named 'SystemCode'

- PYTHONPATH must point to the parent of SystemCode/ (i.e., IRS-PM-2026-01-24-GRP7-WellnessBot/).
- On macOS/Linux, use PYTHONPATH=.. when running commands manually.
- On Windows, run_streamlit.ps1 sets this automatically — use the script rather than running streamlit directly.

7.5 openai.AuthenticationError or 401 Unauthorized

- Your OPENAI_API_KEY in SystemCode/.env is missing or incorrect.
- Verify the key at platform.openai.com.

7.6 Profile not saving

- Ensure the SystemCode/data/profiles/ directory exists. It is created automatically on first run.
- Check write permissions on the SystemCode/data/ directory.

7.7 Logs not being written

- Ensure LOGGING_ENABLED=1 in SystemCode/.env.
- Ensure the SystemCode/logs/ directory exists and is writable.

Appendix D – List of Tested Scenario

No.	Phase	Symptom Response	Pain Score	Swelling Score	Actual Action	Predicted Action
1	P1-1	none	pain 0	swelling 1	Recommend	Recommend
2	P1-1	none	1	2	Recommend	Recommend
3	P1-2	none	1	2	Recommend	Recommend
4	P1-2	none	0	0	Recommend	Recommend
5	P2	none	pain 0	swelling 1	Recommend	Recommend
6	P2	none	0	2	Recommend	Recommend
7	P2	none	0	0	Recommend	Recommend
8	P2	none	pain 0	swelling 2	Recommend	Recommend
9	P3	none	0	0	Recommend	Recommend
10	P3	none	1	0	Recommend	Recommend
11	P3	none	0	1	Recommend	Recommend
12	P4	none	0	0	Recommend	Recommend
13	P4	none	0	2	Recommend	Recommend
14	P1-2	none	1	0	Recommend	Recommend
15	P3	none	pain 1	swelling 2	Recommend	Recommend
16	P4	none	pain 1	1	Recommend	Recommend
17	P1-1	I am running a high fever today.	0	2	Forbid	Forbid
18	P1-2	One of the scars split open a little. There was some fresh spotting of blood on my compression bandage. It stopped within the hour but gave me a fright.	2	0	Forbid	Forbid
19	P1-2	There is some slight bleeding when I change the bandage.	2	0	Forbid	Forbid
20	P2	none	pain 2	swelling 0	Forbid	Forbid
21	P3	none	2	1	Forbid	Forbid
22	P4	none	2	1	Forbid	Forbid
23	P1-1	I noticed a yellowish discharge from one of the portal wounds and ran a low fever of 37.9 °C	0	1	Forbid	Forbid
24	P1-1	none	3	3	Escalate	Escalate
25	P1-2	I have noticed quite a bit of bleeding from the incision.	2	1	Escalate	Forbid
26	P2	none	pain 1	Severe (3)	Escalate	Escalate
27	P2	Spiked a high fever — 39.2 °C — and the joint swelled enormously overnight.	NA	NA	Escalate	Escalate
28	P3	I have a mild fever.	NA	NA	Escalate	Escalate
29	P3	I felt burning hot last night.	NA	NA	Escalate	Escalate
30	P4	I felt mildly feverish — around 38.0 °C — and the knee was warm to touch.	NA	NA	Escalate	Escalate
31	P4	My temperature is over 101 degrees.	NA	NA	Escalate	Escalate
32	P1-1	There was quite a lot of blood seeping from the small portal cuts.	2	1	Escalate	Forbid

Appendix E – Full Self-Care Evaluation Results (Filtered Valid Scenarios)

No.	Phase	Expected Self-Care	Predicted Self-Care	Result
1	P1-1	Prescribed medicine + Ice 20 mins hourly	Prescribed medicine + Ice 20 mins hourly	Pass
2	P1-1	Ice 20 mins hourly	Ice 20 mins hourly	Pass
3	P1-1	Prescribed medicine + Ice 20 mins hourly	Prescribed medicine + Ice 20 mins hourly	Pass
4	P1-2	Prescribed pain relief + Ice 20 mins 3-4times per day	Prescribed pain relief + Ice 20 mins 3-4times per day	Pass
5	P1-2	Prescribed pain relief + Ice 20 mins 3-4times per day	Prescribed pain relief + Ice 20 mins 3-4times per day	Pass
6	P1-2	Ice 20 mins 3-4times per day	Ice 20 mins 3-4times per day	Pass
7	P1-2	Prescribed pain relief + Ice 20 mins 3-4times per day	Prescribed pain relief + Ice 20 mins 3-4times per day	Pass
8	P2	Ice 10 mins when needed / after exercise	Ice 10 mins when needed / after exercise	Pass
9	P2	Ice 20 mins when needed / after exercise	Ice 20 mins when needed / after exercise	Pass
10	P2	Ice 20 mins when needed / after exercise	Ice 20 mins when needed / after exercise	Pass
11	P2	Prescribed pain relief	Prescribed pain relief	Pass
12	P3	Hot towel 10 mins	Hot towel 10 mins	Pass
13	P3	Prescribed pain relief + hot towel 10 mins	Prescribed pain relief + hot towel 10 mins	Pass
14	P3	Ice 10 mins when needed / after exercise	Ice 10 mins when needed / after exercise	Pass
15	P3	Prescribed pain relief + Ice 10 mins when needed / after exercise	Prescribed pain relief + Ice 10 mins when needed / after exercise	Pass
16	P4	Walk 10 minutes before exercise.	Walk 10 minutes before exercise.	Pass
17	P4	Prescribed pain relief + Ice 10 mins when needed / after exercise	Prescribed pain relief + Ice 10 mins when needed / after exercise	Pass
18	P4	Ice 20 mins when needed / after exercise	Ice 20 mins when needed / after exercise	Pass
19	P1-1	Prescribed medicine + Ice 20 mins hourly	Prescribed medicine + Ice 20 mins hourly	Pass
20	P1-2	Prescribed pain relief + Ice 20 mins 3-4times per day	Prescribed pain relief + Ice 20 mins 3-4times per day	Pass
21	P3	Prescribed pain relief + Ice 20 mins when needed / after exercise	Prescribed pain relief + Ice 20 mins when needed / after exercise	Pass
22	P4	Prescribed pain relief + Ice 10 mins when needed / after exercise	Prescribed pain relief + Ice 10 mins when needed / after exercise	Pass